Guideline 8: Compartmentalize your assets

Compartmentalizing means that you should not provide all-or-nothing access to information in a system. Rather, you should organize the information in a system into compartments. Users should only have access to the information that they need, rather than to all of the information in a system. This means that the effects of an attack may be contained. Some information may be lost or damaged but it is unlikely that all of the information in the system will be affected.

For example, in the patient information system, you should design the system so that at any one clinic, the clinic staff normally only have access to the records of patients that have an appointment at that clinic. They should not normally have access to all patient records in the system. Not only does this limit the potential loss from insider attacks, it also means that if an intruder steals their credentials, then the amount of damage that they can cause is limited.

Having said this, you also may have to have mechanisms in the system to grant unexpected access—say to a patient who is seriously ill and requires urgent treatment without an appointment. In those circumstances, you might use some alternative secure mechanism to override the compartmentalization in the system. In such situations, where security is relaxed to maintain system availability, it is essential that you use a logging mechanism to record system usage. You can then check the logs to trace any unauthorized use.

Guideline 9: Design for deployment

Many security problems arise because the system is not configured correctly when it is deployed in its operational environment. You should therefore always design your system so that facilities are included to simplify deployment in the customer's environment and to check for potential configuration errors and omissions in the deployed system. This is an important topic, which I cover in detail later in Section 14.2.3.

Guideline 10: Design for recoverability

Irrespective of how much effort you put into maintaining systems security, you should always design your system with the assumption that a security failure could occur. Therefore, you should think about how to recover from possible failures and restore the system to a secure operational state. For example, you may include a backup authentication system in case your password authentication is compromised.

For example, say an unauthorized person from outside the clinic gains access to the patient records system and you don't know how they obtained a valid login/password combination. You need to reinitialize the authentication system and not just change the credentials used by the intruder. This is essential because the intruder may also have gained access to other user passwords. You need, therefore, to ensure that all authorized users change their passwords. You also must ensure that the unauthorized person does not have access to the password changing mechanism.

You therefore have to design your system to deny access to everyone until they have changed their password and to authenticate real users for password change,

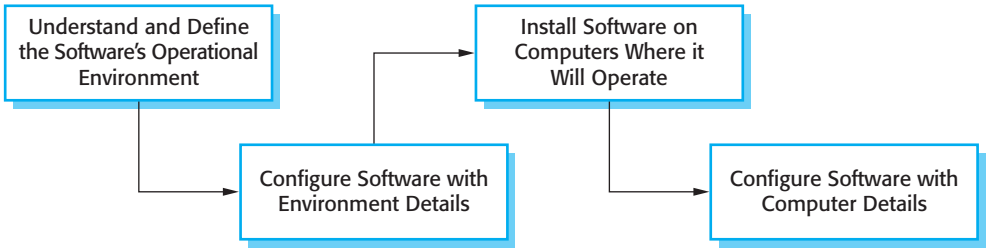


Figure 14.7 Software deployment

assuming that their chosen passwords may not be secure. One way of doing this is to use a challenge/response mechanism, where users have to answer questions for which they have pre-registered answers. This is only invoked when passwords are changed, allowing for recovery from the attack with relatively little user disruption.

14.2.3 Design for deployment

The deployment of a system involves configuring the software to operate in an operational environment, installing the system on the computers in that environment, and then configuring the installed system for these computers (Figure 14.7). Configuration may be a simple process that involves setting some built-in parameters in the software to reflect user preferences. Sometimes, however, configuration is complex and requires the specific definition of business models and rules that affect the execution of the software.

It is at this stage of the software process that vulnerabilities in the software are often accidentally introduced. For example, during installation, software often has to be configured with a list of allowed users. When delivered, this list simply consists of a generic administrator login such as 'admin' and a default password, such as 'password'. This makes it easy for an administrator to set up the system. Their first action should be to introduce a new login name and password, and to delete the generic login name. However, it's easy to forget to do this. An attacker who knows of the default login may then be able to gain privileged access to the system.

Configuration and deployment are often seen as system administration issues and so are considered to be outside the scope of software engineering processes. Certainly, good management practice can avoid many security problems that arise from configuration and deployment mistakes. However, software designers have the responsibility to 'design for deployment'. You should always provide built-in support for deployment that will reduce the probability that system administrators (or users) will make mistakes when configuring the software.

I recommend four ways to incorporate deployment support in a system:

1. *Include support for viewing and analyzing configurations* You should always include facilities in a system that allow administrators or permitted users to examine the current configuration of the system. This facility is, surprisingly, lacking from most software systems and users are frustrated by the difficulties of finding configuration settings. For example, in the version of the word processor that I used to write this chapter, it is impossible to see or print the settings of all system

preferences on a single screen. However, if an administrator can get a complete picture of a configuration, they are more likely to spot errors and omissions. Ideally, a configuration display should also highlight aspects of the configuration that are potentially unsafe—for example, if a password has not been set up.

2. *Minimize default privileges* You should design software so that the default configuration of a system provides minimum essential privileges. This way, the damage that any attacker can do can be limited. For example, the default system administrator authentication should only allow access to a program that enables an administrator to set up new credentials. It should not allow access to any other system facilities. Once the new credentials have been set up, the default login and password should be deleted automatically.

3. *Localize configuration settings* When designing system configuration support, you should ensure that everything in a configuration that affects the same part of a system is set up in the same place. To use the word processor example again, in the version that I use, I can set up some security information, such as a password to control access to the document, using the Preferences/Security menu. Other information is set up in the Tools/Protect Document menu. If configuration information is not localized, it is easy to forget to set it up or, in some cases, not even be aware that some security facilities are included in the system.

4. *Provide easy ways to fix security vulnerabilities* You should include straightforward mechanisms for updating the system to repair security vulnerabilities that have been discovered. These could include automatic checking for security updates, or downloading of these updates as soon as they are available. It is important that users cannot bypass these mechanisms as, inevitably, they will consider other work to be more important. There are several recorded examples of major security problems that arose (e.g., complete failure of a hospital network) because users did not update their software when asked to do so.

14.3 System survivability

So far, I have discussed security engineering from the perspective of an application that is under development. The system procurer and developer have control over all aspects of the system that might be attacked. In reality, as I suggested in Figure 14.1, modern distributed systems inevitably rely on an infrastructure that includes off-the-shelf systems and reusable components that have been developed by different organizations. The security of these systems does not just depend on local design decisions. It is also affected by the security of external applications, web services, and the network infrastructure.

This means that, irrespective of how much attention is paid to security, it cannot be guaranteed that a system will be able to resist external attacks. Consequently, for complex networked systems, you should assume that penetration is possible and that the integrity of the system cannot be guaranteed. You should therefore think about how to make the system resilient so that it survives to deliver essential services to users.

Survivability or resilience (Westmark, 2004) is an emergent property of a system as a whole, rather than a property of individual components, which may not themselves be survivable. The survivability of a system reflects its ability to continue to deliver essential business or mission-critical services to legitimate users while it is under attack or after part of the system has been damaged. The damage could be caused by an attack or by a system failure.

Work on system survivability was prompted by the fact that our economic and social lives are dependent on a computer-controlled critical infrastructure. This includes the infrastructure for delivering utilities (power, water, gas, etc.) and, equally critically, the infrastructure for delivering and managing information (telephones, Internet, postal service, etc.). However, survivability is not simply a critical infrastructure issue. Any organization that relies on critical networked computer systems should be concerned with how its business would be affected if their systems did not survive a malicious attack or catastrophic system failure. Therefore, for business critical systems, survivability analysis and design should be part of the security engineering process.

Maintaining the availability of critical services is the essence of survivability. This means that you have to know:

- the system services that are the most critical for a business;
- the minimal quality of service that must be maintained;
- how these services might be compromised;
- how these services can be protected;
- how you can recover quickly if the services become unavailable.

For example, in a system that handles ambulance dispatch in response to emergency calls, the critical services are those concerned with taking calls and dispatching ambulances to the medical emergency. Other services, such as call logging and ambulance location management, are less critical, either because they do not require real-time processing or because alternative mechanisms may be used. For example, to find an ambulance's location you can call the ambulance crew and ask them where they are.

Ellison and colleagues (1999a; 1999b; 2002) have designed a method of analysis called Survivable Systems Analysis. This is used to assess vulnerabilities in systems and to support the design of system architectures and features that promote system survivability. They argue that achieving survivability depends on three complementary strategies:

1. *Resistance* Avoiding problems by building capabilities into the system to repel attacks. For example, a system may use digital certificates to authenticate users, thus making it more difficult for unauthorized users to gain access.
2. *Recognition* Detecting problems by building capabilities into the system to detect attacks and failures and assess the resultant damage. For example, checksums may be associated with critical data so that corruptions to that data can be detected.
3. *Recovery* Tolerating problems by building capabilities into the system to deliver essential services while under attack, and to recover full functionality after an

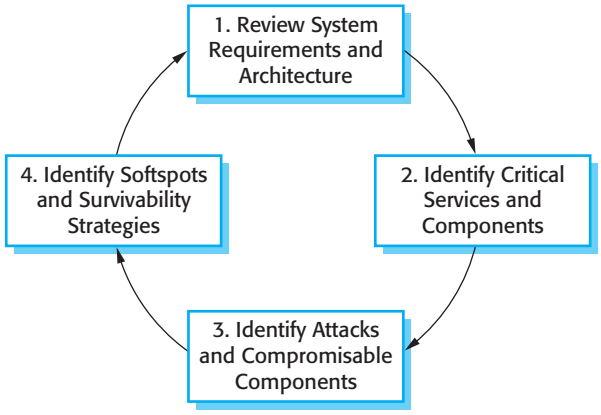


Figure 14.8 Stages in survivability analysis

attack. For example, fault tolerance mechanisms using diverse implementations of the same functionality may be included to cope with a loss of service from one part of the system.

Survivable systems analysis is a four-stage process (Figure 14.8) that analyzes the current or proposed system requirements and architecture; identifies critical services, attack scenarios, and system 'softspots'; and proposes changes to improve the survivability of a system. The key activities in each of these stages are as follows:

1. *System understanding* For an existing or proposed system, review the goals of the system (sometimes called the mission objectives), the system requirements, and the system architecture.
2. *Critical service identification* The services that must always be maintained and the components that are required to maintain these services are identified.
3. *Attack simulation* Scenarios or use cases for possible attacks are identified along with the system components that would be affected by these attacks.
4. *Survivability analysis* Components that are both essential and compromisable by an attack are identified and survivability strategies based on resistance, recognition, and recovery are identified.

Ellison and his colleagues present an excellent case study of the method based on a system to support mental health treatment (1999b). This system is similar to the MHC-PMS that I have used as an example in this book. Rather than repeat their analysis, I use the equity trading system, as shown in Figure 14.5, to illustrate some of the features of survivability analysis.

As you can see from Figure 14.5, this system already has already made some provision for survivability. User accounts and equity prices are replicated across servers so that orders can be placed even if the local server is unavailable. Let's assume that the capability for authorized users to place orders for stock is the key service that must be maintained. To ensure that users trust the system, it is essential that integrity be maintained. Orders must be accurate and reflect the actual sales or purchases made by a system user.

Attack	Resistance	Recognition	Recovery
Unauthorized user places malicious orders	Require a dealing password that is different from the login password to place orders.	Send copy of order by e-mail to authorized user with contact phone number (so that they can detect malicious orders). Maintain user's order history and check for unusual trading patterns.	Provide mechanism to automatically 'undo' trades and restore user accounts. Refund users for losses that are due to malicious trading. Insure against consequential losses.
Corruption of transactions database	Require privileged users to be authorized using a stronger authentication mechanism, such as digital certificates.	Maintain read-only copies of transactions for an office on an international server. Periodically compare transactions to check for corruption. Maintain cryptographic checksum with all transaction records to detect corruption.	Recover database from backup copies. Provide a mechanism to replay trades from a specified time to re-create the transactions database.

Figure 14.9 Survivability analysis in an equity trading system

To maintain this ordering service, there are three components of the system that are used:

1. *User authentication* This allows authorized users to log on to the system.
2. *Price quotation* This allows the buying and selling price of a stock to be quoted.
3. *Order placement* This allows buy and sell orders at a given price to be made.

These components obviously make use of essential data assets such as a user account database, a price database, and an order transaction database. These must survive attacks if service is to be maintained.

There are several different types of attack on this system that might be made. Let's consider two possibilities here:

1. A malicious user has a grudge against an accredited system user. He gains access to the system using their credentials. Malicious orders are placed and stock is bought and sold, with the intention of causing problems for the authorized user.
2. An unauthorized user corrupts the database of transactions by gaining permission to issue SQL commands directly. Reconciliation of sales and purchases is therefore impossible.

Figure 14.9 shows examples of resistance, recognition, and recovery strategies that might be used to help counter these attacks.

Increasing the survivability or resilience of a system of course costs money. Companies may be reluctant to invest in survivability if they have never suffered a serious attack or associated loss. However, just as it is best to buy good locks and an alarm before rather than after your house is burgled, it is best to invest in survivability before, rather than after, a successful attack. Survivability analysis is not yet part of most software engineering processes but, as more and more systems become business critical, such analyzes are likely to become more widely used.

KEY POINTS

- Security engineering focuses on how to develop and maintain software systems that can resist malicious attacks intended to damage a computer-based system or its data.
- Security threats can be threats to the confidentiality, integrity, or availability of a system or its data.
- Security risk management involves assessing the losses that might ensue from attacks on a system, and deriving security requirements that are aimed at eliminating or reducing these losses.
- Design for security involves designing a secure system architecture, following good practice for secure systems design, and including functionality to minimize the possibility of introducing vulnerabilities when the system is deployed.
- Key issues when designing a secure systems architecture include organizing the system structure to protect key assets and distributing the system assets to minimize the losses from a successful attack.
- Security design guidelines sensitize system designers to security issues that they may not have considered. They provide a basis for creating security review checklists.
- To support secure deployment you should provide a way of displaying and analyzing system configurations, localize configuration settings so that important configurations are not forgotten, minimize default privileges assigned to system users, and provide ways to repair security vulnerabilities.
- System survivability reflects the ability of a system to continue to deliver essential business or mission-critical services to legitimate users while it is under attack, or after part of the system has been damaged.

FURTHER READING

'Survivable Network System Analysis: A Case Study.' An excellent paper that introduces the notion of system survivability and uses a case study of a mental health record treatment system to illustrate the application of a survivability method. (R. J. Ellison, R. C. Linger, T. Longstaff and N. R. Mead, *IEEE Software*, **16** (4), July/August 1999.)

Building Secure Software: How to Avoid Security Problems the Right Way. A good practical book covering security from a programming perspective. (J. Viega and G. McGraw, Addison-Wesley, 2002.)

Security Engineering: A Guide to Building Dependable Distributed Systems, 2nd edition. This is a thorough and comprehensive discussion of the problems of building secure systems. The focus is on systems rather than software engineering with extensive coverage of hardware and networking, with excellent examples drawn from real system failures. (R. Anderson, John Wiley & Sons, 2008.)

EXERCISES

14.1. Explain the important differences between application security engineering and infrastructure security engineering.

14.2. For the MHC-PMS, suggest an example of an asset, exposure, vulnerability, attack, threat, and control.

14.3. Explain why there is a need for risk assessment to be a continuing process from the early stages of requirements engineering through to the operational use of a system.

14.4. Using your answers to question 2 about the MHC-PMS, assess the risks associated with that system and propose two system requirements that might reduce these risks.

14.5. Explain, using an analogy drawn from a non-software engineering context, why a layered approach to asset protection should be used.

14.6. Explain why it is important to use diverse technologies to support distributed systems in situations where system availability is critical.

14.7. What is social engineering? Why is it difficult to protect against it in large organizations?

14.8. For any off-the-shelf software system that you use (e.g., Microsoft Word), analyze the configuration facilities included and discuss any problems that you find.

14.9. Explain how the complementary strategies of resistance, recognition, and recovery may be used to enhance the survivability of a system.

14.10. For the equity trading system discussed in Section 14.2.1, whose architecture is shown in Figure 14.5, suggest two further plausible attacks on the system and propose possible strategies that could counter these attacks.

REFERENCES

Alberts, C. and Dorofee, A. (2002). *Managing Information Security Risks: The OCTAVE Approach.* Boston: Addison-Wesley.

Alexander, I. (2003). 'Misuse Cases: Use Cases with Hostile Intent'. *IEEE Software*, **20** (1), 58–66.

Anderson, R. (2008). *Security Engineering, 2nd edition*. Chichester: John Wiley & Sons.

Berghel, H. (2001). 'The Code Red Worm'. *Comm. ACM*, **44** (12), 15–19.

Bishop, M. (2005). *Introduction to Computer Security*. Boston: Addison-Wesley.

Cranor, L. and Garfinkel, S. (2005). *Security and Usability: Designing secure systems that people can use*. Sebastopol, Calif.: O'Reilly Media Inc.

Ellison, R., Linger, R., Lipson, H., Mead, N. and Moore, A. (2002). 'Foundations of Survivable Systems Engineering'. *Crosstalk: The Journal of Defense Software Engineering*, **12**, 10–15.

Ellison, R. J., Fisher, D. A., Linger, R. C., Lipson, H. F., Longstaff, T. A. and Mead, N. R. (1999a). 'Survivability: Protecting Your Critical Systems'. *IEEE Internet Computing*, **3** (6), 55–63.

Ellison, R. J., Linger, R. C., Longstaff, T. and Mead, N. R. (1999b). 'Survivable Network System Analysis: A Case Study'. *IEEE Software*, 16 (4), 70–7.

Pfleeger, C. P. and Pfleeger, S. L. (2007). *Security in Computing, 4th edition*. Boston: Addison-Wesley.

Schneier, B. (2000). Secrets and Lies: *Digital Security in a Networked World*. New York: John Wiley & Sons.

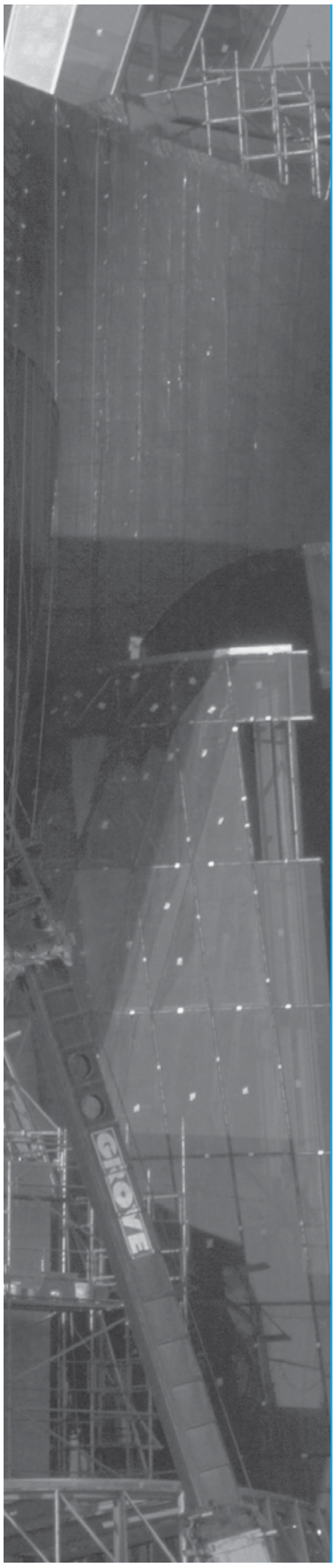
Sindre, G. and Opdahl, A. L. (2005). 'Eliciting Security Requirements through Misuse Cases'. *Requirements Engineering*, **10** (1), 34–44.

Spafford, E. (1989). 'The Internet Worm: Crisis and Aftermath'. *Comm ACM*, **32** (6), 678–87.

Viega, J. and McGraw, G. (2002). *Building Secure Software*. Boston: Addison-Wesley.

Westmark, V. R. (2004). 'A Definition for Information System Survivability'. 37th Hawaii Int. Conf. on System Sciences, Hawaii: 903–1003.

Wheeler, D. A. (2003). *Secure Programming for Linux and UNix HOWTO*. Web published: http://www.dwheeler.com/secure-programs/Secure-Programs-HOWTO/index.html.



15

Dependability and security assurance

Objectives

The objective of this chapter is to describe the verification and validation techniques that are used in the development of critical systems. When you have read this chapter, you will:

- understand how different approaches to static analysis may be used in the verification of critical software systems;
- understand the basics of reliability and security testing and the inherent problems of testing critical systems;
- know why process assurance is important, especially for software that has to be certified by a regulator;
- have been introduced to safety and dependability cases that present arguments and evidence of system safety and dependability.

Contents

15.1 Static analysis
15.2 Reliability testing
15.3 Security testing
15.4 Process assurance
15.5 Safety and dependability cases

Dependability and security assurance is concerned with checking that a critical system meets its dependability requirements. This requires verification and validation (V & V) processes that look for specification, design, and program errors that may affect the availability, safety, reliability, or security of a system.

The verification and validation of a critical system has much in common with the validation of any other software system. The V & V processes should demonstrate that the system meets its specification and that the system services and behavior support the customer's requirements. In doing so, they usually uncover requirements and design errors and program bugs that have to be repaired. However, critical systems require particularly stringent testing and analysis for two reasons:

1. *Costs of failure* The costs and consequences of critical systems failure are potentially much greater than for non-critical systems. You lower the risks of system failure by spending more on system verification and validation. It is usually cheaper to find and remove defects before the system is delivered than to pay for the consequent costs of accidents or disruptions to system service.
2. *Validation of dependability attributes* You may have to make a formal case to customers and a regulator that the system meets its specified dependability requirements (availability, reliability, safety, and security). In some cases, external regulators, such as national aviation authorities, may have to certify that the system is safe before it can be deployed. To obtain this certification, you have to demonstrate how the system has been validated. To do so, you may also have to design and carry out special V & V procedures that collect evidence about the system's dependability.

For these reasons, verification and validation costs for critical systems are usually much higher than for other classes of systems. Typically, more than half of a critical system's development costs are spent on V & V.

Although V & V costs are high, they are justified as they are usually significantly less than the losses that result from an accident. For example, in 1996, a mission-critical software system on the Ariane 5 rocket failed and several satellites were destroyed. No one was injured but the total losses from this accident were hundreds of millions of dollars. The subsequent enquiry discovered that deficiencies in system V & V were partly responsible for this failure. More effective reviews, which would have been relatively cheap, could have discovered the problem that caused the accident.

Although the primary focus of dependability and security assurance is on the validation of the system itself, related activities should verify that the defined system development process has been followed. As I explained in Chapter 13, system quality is affected by the quality of processes used to develop the system. In short, good processes lead to good systems.

The outcome of dependability and security assurance processes is a body of tangible evidence, such as review reports, test results, etc., about the dependability of a system. This evidence may subsequently be used to justify a decision that this system is dependable and secure enough to be deployed and used. Sometimes, the evidence

of system dependability is assembled in a dependability or safety case. This is used to convince a customer or an external regulator that the developer's confidence in the system's dependability or safety is justified.

15.1 Static analysis

Static analysis techniques are system verification techniques that don't involve executing a program. Rather, they work on a source representation of the software—either a model of the specification or design, or the source code of the program. Static analysis techniques can be used to check the specification and design models of a system to pick up errors before an executable version of the system is available. They also have the advantage that the presence of errors does not disrupt system checking. When you test a program, defects can mask or hide other defects so you have to remove a detected defect then repeat the testing process.

As I discussed in Chapter 8, perhaps the most commonly used static analysis technique is peer review and inspection, where a specification, design, or program is checked by a group of people. They examine the design or code in detail, looking for possible errors or omissions. Another technique is using design modeling tools to check for anomalies in the UML, such as the same name being used for different objects. However, for critical systems, additional static analysis techniques may be used:

1. Formal verification, where you produce mathematically rigorous arguments that a program conforms to its specification.
2. Model checking, where a theorem prover is used to check a formal description of the system for inconsistencies.
3. Automated program analysis, where the source code of a program is checked for patterns that are known to be potentially erroneous.

These techniques are closely related. Model checking relies on a formal model of the system that may be created from a formal specification. Static analyzers may use formal assertions embedded in a program as comments to check that the associated code is consistent with these assertions.

15.1.1 Verification and formal methods

Formal methods of software development, as I discussed in Chapter 12, rely on a formal model of the system that serves as a system specification. These formal methods are mainly concerned with a mathematical analysis of the specification; with transforming the specification to a more detailed, semantically equivalent representation; or with formally verifying that one representation of the system is semantically equivalent to another representation.



Cleanroom development

Cleanroom software development is based on formal software verification and statistical testing. The objective of the Cleanroom process is zero-defects software to ensure that delivered systems have a high level of reliability. In the Cleanroom process each software increment is formally specified and this specification is transformed into an implementation. Software correctness is demonstrated using a formal approach. There is no unit testing for defects in the process and the system testing is focused on assessing the system's reliability.

http://www.SoftwareEngineering-9.com/Web/Cleanroom/

Formal methods may be used at different stages in the V & V process:

1. A formal specification of the system may be developed and mathematically analyzed for inconsistency. This technique is effective in discovering specification errors and omissions. Model checking, discussed in the next section, is one approach to specification analysis.
2. You can formally verify, using mathematical arguments, that the code of a software system is consistent with its specification. This requires a formal specification. It is effective in discovering programming and some design errors.

Because of the wide semantic gap between a formal system specification and program code, it is difficult to prove that a separately developed program is consistent with its specification. Work on program verification is now, therefore, based on transformational development. In a transformational development process, a formal specification is transformed through a series of representations to program code. Software tools support the development of the transformations and help verify that corresponding representations of the system are consistent. The B method is probably the most widely used formal transformational method (Abrial, 2005; Wordsworth, 1996). It has been used for the development of train control systems and avionics software.

Proponents of formal methods claim that the use of these methods leads to more reliable and safer systems. Formal verification demonstrates that the developed program meets its specification and that implementation errors will not compromise the dependability of the system. If you develop a formal model of concurrent systems using a specification written in a language such as CSP (Schneider, 1999), you can discover conditions that might result in deadlock in the final program, and be able to address these. This is very difficult to do by testing alone.

However, formal specification and proof do not guarantee that the software will be reliable in practical use. The reasons for this are as follows:

1. The specification may not reflect the real requirements of system users. As I discussed in Chapter 12, system users rarely understand formal notations so they cannot directly read the formal specification to find errors and omissions. This means that there is a significant likelihood that the formal specification contains errors and is not an accurate representation of the system requirements.

2. The proof may contain errors. Program proofs are large and complex, so, like large and complex programs, they usually contain errors.
3. The proof may make incorrect assumptions about the way that the system is used. If the system is not used as anticipated, the proof may be invalid.

Verifying a non-trivial software system takes a great deal of time and requires mathematical expertise and specialized software tools, such as theorem provers. It is therefore an expensive process and, as the system size increases, the costs of formal verification increase disproportionately. Many software engineers therefore think that formal verification is not cost effective. They believe that the same level of confidence in the system can be achieved more cheaply by using other validation techniques, such as inspections and system testing.

In spite of their disadvantages, my view is that formal methods and formal verification have an important role to play in the development of critical software systems. Formal specifications are very effective in discovering those specification problems that are the most common causes of system failure. Although formal verification is still impractical for large systems, it can be used to verify critical-safety and security-critical components.

15.1.2 Model checking

Formally verifying programs using a deductive approach is difficult and expensive but alternative approaches to formal analysis have been developed that are based on a more restricted notion of correctness. The most successful of these approaches is called model checking (Baier and Katoen, 2008). This has been widely used to check hardware systems designs and is increasingly being used in critical software systems such as the control software in NASA's Mars exploration vehicles (Regan and Hamilton, 2004) and telephone call processing software (Chandra et al., 2002).

Model checking involves creating a model of a system and checking the correctness of that model using specialized software tools. Many different model-checking tools have been developed—for software, the most widely used is probably SPIN (Holzmann, 2003). The stages involved in model checking are shown in Figure 15.1.

The model-checking process involves building a formal model of a system, usually as an extended finite state machine. Models are expressed in the language of whatever model-checking system is used—for example, the SPIN model checker uses a language called Promela. A set of desirable system properties are identified and written in a formal notation, usually based on temporal logic. An example of such a property in the wilderness weather system might be that the system will always reach the 'transmitting' state from the 'recording' state.

The model checker then explores all paths through the model (i.e., all possible state transitions), checking that the property holds for each path. If it does, then the model checker confirms that the model is correct with respect to that property. If it does not hold for a particular path, the model checker outputs a counter-example illustrating where the property is not true. Model checking is particularly useful in

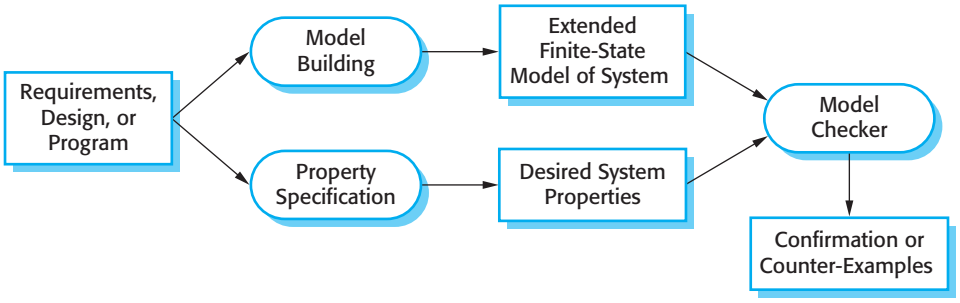


Figure 15.1 Model checking

the validation of concurrent systems, which are notoriously difficult to test because of their sensitivity to time. The checker can explore interleaved, concurrent transitions and discover potential problems.

A key issue in model checking is the creation of the system model. If the model has to be created manually (from a requirements or design document), it is an expensive process as model creation takes a great deal of time. In addition, there is the possibility that the model created will not be an accurate model of the requirements or design. It is, therefore, best if the model can be created automatically from the program source code. The Java Pathfinder system (Visser et al., 2003) is an example of a model-checking system that works directly from a representation of Java code.

Model checking is computationally very expensive because it uses an exhaustive approach to check all paths through the system model. As the size of the system increases, so too does the number of states, with a consequent increase in the number of paths to be checked. This means that, for large systems, model checking may be impractical, due to the computer time required to run the checks.

However, as algorithms for identifying those parts of the state that do not have explored to check a particular property improve, it will become increasingly practical to use model-checking routinely in critical systems development. It is not really applicable to data-oriented organizational systems, but it can be used to verify embedded software systems that are modeled as state machines.

15.1.3 Automatic static analysis

As I discussed in Chapter 8, program inspections are often driven by checklists of errors and heuristics. These identify common errors in different programming languages. For some errors and heuristics, it is possible to automate the process of checking programs against these lists, which has resulted in the development of automated static analyzers that can find code fragments that may be incorrect.

Static analysis tools work on the source code of a system and, for some types of analysis at least, no further inputs are required. This means that programmers do not need to learn specialized notations to write program specifications so the benefits of analysis can be immediately clear. This makes automated static analysis easier to introduce into a development process than formal verification or model checking. It is, therefore, probably the most widely used static analysis technique.

Fault class	Static analysis check
Data faults	Variables used before initialization Variables declared but never used Variables assigned twice but never used between assignments Possible array bound violations Undeclared variables
Control faults	Unreachable code Unconditional branches into loops
Input/output faults	Variables output twice with no intervening assignment
Interface faults	Parameter-type mismatches Parameter number mismatches Non-usage of the results of functions Uncalled functions and procedures
Storage management faults	Unassigned pointers Pointer arithmetic Memory leaks

Figure 15.2 Automated static analysis checks

Automated static analyzers are software tools that scan the source text of a program and detect possible faults and anomalies. They parse the program text and thus recognize the different types of statements in a program. They can then detect whether or not statements are well formed, make inferences about the control flow in the program, and, in many cases, compute the set of all possible values for program data. They complement the error detection facilities provided by the language compiler, and can be used as part of the inspection process or as a separate V & V process activity. Automated static analysis is faster and cheaper than detailed code reviews. However, it cannot discover some classes of errors that could be identified in program inspection meetings.

The intention of automatic static analysis is to draw a code reader's attention to anomalies in the program, such as variables that are used without initialization, variables that are unused, or data whose value could go out of range. Examples of the problems that can be detected by static analysis are shown in Figure 15.2. Of course, the specific checks made are programming-language specific and depend on what is and isn't allowed in the language. Anomalies are often a result of programming errors or omissions, so they highlight things that could go wrong when the program is executed. However, you should understand that these anomalies are not necessarily program faults; they may be deliberate constructs introduced by the programmer, or the anomaly may have no adverse consequences.

There are three levels of checking that may be implemented in static analyzers:

1. *Characteristic error checking* At this level, the static analyzer knows about common errors that are made by programmers in languages such as Java or C. The tool analyzes the code looking for patterns that are characteristic of that

problem and highlights these to the programmer. Although relatively simple, analysis based on common errors can be very cost effective. Zheng and his collaborators (2006) studied the use of static analysis against a large code base in C and C++ and discovered that 90% of the errors in the programs resulted from 10 types of characteristic error.

2. *User-defined error checking* In this approach, the users of the static analyzer may define error patterns, thus extending the types of error that may be detected. This is particularly useful in situations where ordering must be maintained (e.g., method A must always be called before method B). Over time, an organization can collect information about common bugs that occur in their programs and extend the static analysis tools to highlight these errors.

3. *Assertion checking* This is the most general and most powerful approach to static analysis. Developers include formal assertions (often written as stylized comments) in their program that state relationships that must hold at that point in a program. For example, an assertion might be included that states that the value of some variable must lie in the range x..y. The analyzer symbolically executes the code and highlights statements where the assertion may not hold. This approach is used in analyzers such as Splint (Evans and Larochelle, 2002) and the SPARK Examiner (Croxford and Sutton, 2006).

Static analysis is effective in finding errors in programs but commonly generates a large number of 'false positives'. These are code sections where there are no errors but where the static analyzer's rules have detected a potential for errors. The number of false positives can be reduced by adding more information to the program in the form of assertions but, obviously, this requires additional work by the developer of the code. Work has to be done in screening out these false positives before the code itself can be checked for errors.

Static analysis is particularly valuable for security checking (Evans and Larochelle, 2002). Static analyzers can be tailored to check for well-known problems, such as buffer overflow or unchecked inputs, which can be exploited by attackers. Checking for well-known problems is effective for improving security as most attackers base their attacks on common vulnerabilities.

As I discuss later, security testing is difficult because attackers often do unexpected things that testers find difficult to anticipate. Static analyzers can incorporate detailed security expertise that testers may not have and may be applied before a program is tested. If you use static analysis, you can make claims that are true for all possible program executions, not just those that correspond to the tests that you have designed.

Static analysis is now routinely used by many organizations in their software development processes. Microsoft introduced static analysis in the development of device drivers (Larus, et al., 2003) where program failures can have a serious effect. They have now extended the approach across a much wider range of their software to look for security problems as well as errors that affect program reliability (Ball, et al., 2006). Many critical systems, including avionics and nuclear systems, are routinely statically analyzed as part of the V & V process (Nguyen and Ourghanlian, 2003).

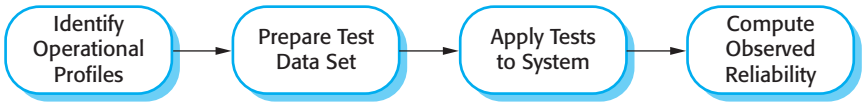


Figure 15.3 Reliability measurement

15.2 Reliability testing

Reliability testing is a testing process that aims to measure the reliability of a system. As I explained in Chapter 10, there are several reliability metrics such as POFOD, probability of failure on demand and ROCOF, the rate of occurrence of failure. These may be used to quantitatively specify the required software reliability. You can check in the reliability testing process if the system has achieved that required reliability level.

The process of measuring the reliability of a system is illustrated in Figure 15.3. This process involves four stages:

1. You start by studying existing systems of the same type to understand how these are used in practice. This is important as you are trying to measure the reliability as it is seen by a system user. Your aim is to define an operational profile. An operational profile identifies classes of system inputs and the probability that these inputs will occur in normal use.
2. You then construct a set of test data that reflects the operational profile. This means that you create test data with the same probability distribution as the test data for the systems that you have studied. Normally, you will use a test data generator to support this process.
3. You test the system using these data and count the number and type of failures that occur. The times of these failures are also logged. As I discussed in Chapter 10, the time units chosen should be appropriate for the reliability metric used.
4. After you have observed a statistically significant number of failures, you can compute the software reliability and work out the appropriate reliability metric value.

This four-step approach is sometimes called 'statistical testing'. The aim of statistical testing is to assess system reliability. This contrasts with defect testing, discussed in Chapter 8, where the aim is to discover system faults. Prowell et al. (1999) give a good description of statistical testing in their book on Cleanroom software engineering.

This conceptually attractive approach to reliability measurement is not easy to apply in practice. The principal difficulties that arise are:

1. *Operational profile uncertainty* The operational profiles based on experience with other systems may not be an accurate reflection of the real use of the system.
2. *High costs of test data generation* It can be very expensive to generate the large volume of data required in an operational profile unless the process can be totally automated.

3. *Statistical uncertainty when high reliability is specified* You have to generate a statistically significant number of failures to allow accurate reliability measurements. When the software is already reliable, relatively few failures occur and it is difficult to generate new failures.

4. *Recognizing failure* It is not always obvious whether or not a system failure has occurred. If you have a formal specification, you may be able to identify deviations from that specification but, if the specification is in natural language, there may be ambiguities that mean observers could disagree on whether the system has failed.

By far the best way to generate the large data set required for reliability measurement is to use a test data generator, which can be set up to automatically generate inputs matching the operational profile. However, it is not usually possible to automate the production of all test data for interactive systems because the inputs are often a response to system outputs. Data sets for these systems have to be generated manually, with correspondingly higher costs. Even where complete automation is possible, writing commands for the test data generator may take a significant amount of time.

Statistical testing may be used in conjunction with fault injection to gather data about how effective the process of defect testing has been. Fault injection (Voas, 1997) is the deliberate injection of errors into a program. When the program is executed, these lead to program faults and associated failures. You then analyze the failure to discover if the root cause is one the errors that you have added to the program. If you find that X% of the injected faults lead to failures, then proponents of fault injection argue that this suggests that the defect testing process will also have discovered X% of the actual faults in the program.

This, of course, assumes that the distribution and type of injected faults matches the actual faults that arise in practice. It is reasonable to think that this might be true for faults due to programming errors, but fault injection is not effective in predicting the number of faults that stem from requirements or design errors.

Statistical testing often reveals errors in the software that have not been discovered by other V & V processes. These errors may mean that a system's reliability falls short of requirements and repairs have to be made. After these repairs are complete, the system can be retested to reassess its reliability. After this repair and retest process has been repeated several times, it may be possible to extrapolate the results and predict when some required level of reliability will be achieved. This requires fitting the extrapolated data to a reliability growth model, which shows how reliability tends to improve over time. This helps with the planning of testing. Sometimes, a growth model may reveal that a required level of reliability will never be achieved, so the requirements have to be renegotiated.

15.2.1 Operational profiles

The operational profile of a software system reflects how it will be used in practice. It consists of a specification of classes of input and the probability of their occurrence. When a new software system replaces an existing automated system, it is



Reliability growth modeling

A reliability growth model is a model of how the system reliability changes over time during the testing process. As system failures are discovered, the underlying faults causing these failures are repaired so that the reliability of the system should improve during system testing and debugging. To predict reliability, the conceptual reliability growth model must then be translated into a mathematical model.

http://www.SoftwareEngineering-9.com/Web/DepSecAssur/RGM.html

reasonably easy to assess the probable pattern of usage of the new software. It should correspond to the existing usage, with some allowance made for the new functionality that is (presumably) included in the new software. For example, an operational profile can be specified for telephone switching systems because telecommunication companies know the call patterns that these systems have to handle.

Typically, the operational profile is such that the inputs that have the highest probability of being generated fall into a small number of classes, as shown on the left of Figure 15.4. There is a very large number of classes where inputs are highly improbable but not impossible. These are shown on the right of Figure 15.4. The ellipsis (. . .) means that there are many more of these unusual inputs than are shown.

Musa (1998) discusses the development of operational profiles in telecommunication systems. As there is a long history of collecting usage data in that domain, the process of operational profile development is relatively straightforward. It simply reflects the historical usage data. For a system that required about 15 person-years of development effort, an operational profile was developed in about 1 person-month. In other cases, operational profile generation took longer (2–3 person-years) but the cost was spread over a number of system releases. Musa reckons that his company had at least a 10-fold return on the investment required to develop an operational profile.

However, when a software system is new and innovative, it is difficult to anticipate how it will be used. Consequently, it is practically impossible to create an accurate operational profile. Many different users with different expectations, backgrounds, and experience may use the new system. There is no historical usage database. These users may make use of systems in ways that were not anticipated by the system developers.

Developing an accurate operational profile is certainly possible for some types of system, such as telecommunication systems, that have a standardized pattern of use. For other system types, however, there are many different users who each have their own ways of using the system. As I discussed in Chapter 10, different users can get quite different impressions of reliability because they use the system in different ways.

The problem is further compounded because operational profiles are not static but change as the system is used. As users learn about a new system and become more confident with it, they start to use it in more sophisticated ways. Because of this, it is often impossible to develop a trustworthy operational profile. Consequently, you cannot be confident about the accuracy of any reliability measurements, as they may be based on incorrect assumptions about the ways in which the system is used.

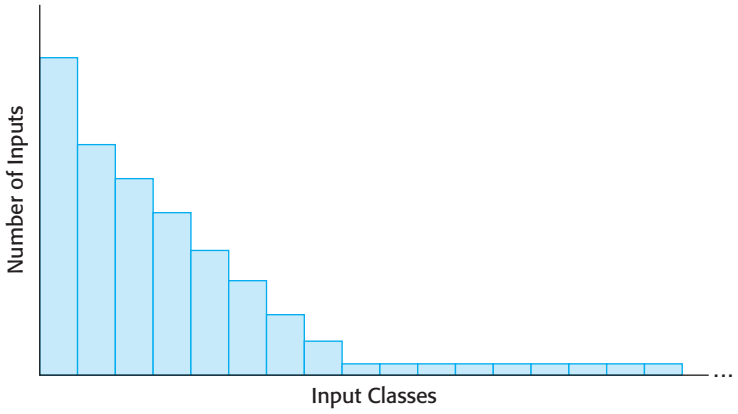


Figure 15.4 An operational profile

15.3 Security testing

The assessment of system security is increasingly important as more and more critical systems are Internet-enabled and so can be accessed by anyone with a network connection. There are daily stories of attacks on web-based systems, and viruses and worms are regularly distributed using Internet protocols.

All of this means that the verification and validation processes for web-based systems must focus on security assessment, where the ability of the system to resist different types of attack is tested. However, as Anderson explains (2001), this type of security assessment is very difficult to carry out. Consequently, systems are often deployed with security loopholes. Attackers use these to gain access to the system or to cause damage to the system or its data.

Fundamentally, there are two reasons why security testing is so difficult:

1. Security requirements, like some safety requirements, are 'shall not' requirements. That is, they specify what should not happen rather than system functionality or required behavior. It is not usually possible to define this unwanted behavior as simple constraints to be checked by the system.

   If resources are available, you can demonstrate, in principle at least, that a system meets its functional requirements. However, it is impossible to prove that a system does not do something. Irrespective of the amount of testing, security vulnerabilities may remain in a system after it has been deployed. You may, of course, generate functional requirements that are designed to guard the system against some known types of attack. However, you cannot derive requirements for unknown or unanticipated types of attack. Even in systems that have been in use for many years, an ingenious attacker can discover a new form of attack and can penetrate what was thought to be a secure system.

2. The people attacking a system are intelligent and are actively looking for vulnerabilities that they can exploit. They are willing to experiment with the system and to try things that are far outside normal activity and system use. For example, in a surname field they may enter 1,000 characters with a mixture of letters, punctuation, and numbers. Furthermore, once they find a vulnerability, they can exchange information about this and so increase the number of potential attackers.

Attackers may try to discover the assumptions made by system developers and then contradict these assumptions to see what happens. They are in a position to use and explore a system over a period of time and analyze it using software tools to discover vulnerabilities that they may be able to exploit. They may, in fact, have more time to spend on looking for vulnerabilities than system test engineers, as testers must also focus on testing the system.

For this reason, static analysis can be particularly useful as a security testing tool. A static analysis of a program can quickly guide the testing team to areas of a program that may include errors and vulnerabilities. Anomalies revealed in the static analysis can be directly fixed or can help identify tests that need to be done to reveal whether or not these anomalies actually represent a risk to the system.

To check the security of a system, you can use a combination of testing, tool-based analysis, and formal verification:

1. *Experience-based testing* In this case, the system is analyzed against types of attack that are known to the validation team. This may involve developing test cases or examining the source code of a system. For example, to check that the system is not susceptible to the well-known SQL poisoning attack, you might test the system using inputs that include SQL commands. To check that buffer overflow errors will not occur, you can examine all input buffers to see if the program is checking that assignments to buffer elements are within bounds.

   This type of validation is usually carried out in conjunction with tool-based validation, where the tool gives you information that helps focus system testing. Checklists of known security problems may be created to assist with the process. Figure 15.5 gives some examples of questions that might be used to drive experience-based testing. Checks on whether the design and programming guidelines for security (Chapter 14) have been followed might also be included in a security problem checklist.

2. *Tiger teams* This is a form of experience-based testing where it is possible to draw on experience from outside the development team to test an application system. You set up a 'tiger team' who are given the objective of breaching the system security. They simulate attacks on the system and use their ingenuity to discover new ways to compromise the system security. Tiger team members should have previous experience with security testing and finding security weaknesses in systems.

3. *Tool-based testing* For this method, various security tools such as password checkers are used to analyze the system. Password checkers detect insecure passwords such as common names or strings of consecutive letters. This

Security checklist
1. Do all files that are created in the application have appropriate access permissions? The wrong access permissions may lead to these files being accessed by unauthorized users.
2. Does the system automatically terminate user sessions after a period of inactivity? Sessions that are left active may allow unauthorized access through an unattended computer.
3. If the system is written in a programming language without array bound checking, are there situations where buffer overflow may be exploited? Buffer overflow may allow attackers to send code strings to the system and then execute them.
4. If passwords are set, does the system check that passwords are 'strong'? Strong passwords consist of mixed letters, numbers, and punctuation, and are not normal dictionary entries. They are more difficult to break than simple passwords.
5. Are inputs from the system's environment always checked against an input specification? Incorrect processing of badly formed inputs is a common cause of security vulnerabilities.

Figure 15.5 Examples of entries in a security checklist

approach is really an extension of experience-based validation, where experience of security flaws is embodied in the tools used. Static analysis is, of course, another type of tool-based testing.

4. *Formal verification* A system can be verified against a formal security specification. However, as in other areas, formal verification for security is not widely used.

Security testing is, inevitably, limited by the time and resources available to the test team. This means that you should normally adopt a risk-based approach to security testing and focus on what you think are the most significant risks faced by the system. If you have an analysis of the security risks to the system, these can be used to drive the testing process. As well as testing the system against the security requirements derived from these risks, the test team should also try to break the system by adopting alternative approaches that threaten the system assets.

It is very difficult for end-users of a system to verify its security. Consequently, government bodies in North America and in Europe have established sets of security evaluation criteria that can be checked by specialized evaluators (Pfleeger and Pfleeger, 2007). Software product suppliers can submit their products for evaluation and certification against these criteria. Therefore, if you have a requirement for a particular level of security, you can choose a product that has been validated to that level. In practice, however, these criteria have primarily been used in military systems and as of yet have not achieved much commercial acceptance.

15.4 Process assurance

As I discussed in Chapter 13, experience has shown that dependable processes lead to dependable systems. That is, if a process is based on good software engineering practices, then it is more likely that the resulting software product will be dependable.



Regulation of software

Regulators are created by governments to ensure that private industry does not profit by failing to follow national standards for safety, security, and so on. There are regulators in many different industries such as nuclear power, aviation, and banking. As software systems have become increasingly important in the critical infrastructure of countries, these regulators have become increasingly concerned with safety and dependability cases for software systems.

http://www.SoftwareEngineering-9.com/Web/DepSecAssur/Regulation.html

Of course, a good process does not guarantee dependability. However, evidence that a dependable process has been used increases overall confidence that a system is dependable. Process assurance is concerned with collecting information about processes used during system development, and the outcomes of these processes. This information provides evidence of the analyses, reviews, and tests that have been carried out during software development.

Process assurance is concerned with two things:

1. Do we have the right processes? Do the system development processes used in the organization include appropriate controls and V & V subprocesses for the type of system being developed?
2. Are we doing the processes right? Has the organization carried out the development work as defined in its software process descriptions and have the defined outcomes from the software processes been produced?

Companies that have extensive experience of critical systems engineering have evolved their processes to reflect good verification and validation practice. In some cases, this has involved discussions with the external regulator to agree on what processes should be used. Although there is a great deal of process variation between companies, activities that you would expect to see in critical systems development processes include requirements management, change management and configuration control, system modeling, reviews and inspections, test planning, and test coverage analysis. The notion of process improvement, where good practice is introduced and institutionalized in processes, is covered in Chapter 26.

The other aspect of process assurance is checking that processes have been properly enacted. This normally involves ensuring that processes are properly documented and checking this process documentation. For example, part of a dependable process may involve formal program inspections. The documentation for each inspection should include the checklists used to drive the inspection, a list of the people involved, the problems identified during the inspection, and the actions required.

Demonstrating that a dependable process has been used therefore involves producing a lot of documentary evidence about the process and the software being developed. The need for this extensive documentation means that agile processes are



Licensing of software engineers

In some areas of engineering, safety engineers must be licensed engineers. Inexperienced, poorly qualified engineers are not allowed to take responsibility for safety. This does not currently apply to software engineers, although there has been extensive discussion on the licensing of software engineers in several states in the United States (Knight and Leveson, 2002). However, future process standards for safety-critical software development may require that project safety engineers should be licensed engineers, with a defined minimum level of qualifications and experience.

http://www.SoftwareEngineering-9.com/Web/DepSecAssur/Licensing.html

rarely used in systems where safety or dependability certification is required. Agile processes focus on the software itself and (rightly) argue that a great deal of process documentation is never actually used after it has been produced. However, you have to create evidence and document process activities when process information is used as part of a system safety or dependability case.

15.4.1 Processes for safety assurance

Most work on process assurance has been done in the area of safety-critical systems development. It is important that a safety-critical systems development process include V & V processes that are geared to safety analysis and assurance for two reasons:

1. Accidents are rare events in critical systems and it may be practically impossible to simulate them during the testing of a system. You can't rely on extensive testing to replicate the conditions that can lead to an accident.
2. Safety requirements, as I discussed in Chapter 12, are sometimes 'shall not' requirements that exclude unsafe system behavior. It is impossible to demonstrate conclusively through testing and other validation activities that these requirements have been met.

Specific safety assurance activities should be included at all stages in the software development process. These safety assurance activities record the analyses that have been carried out and the person or people responsible for these analyses. Safety assurance activities that are incorporated into software processes may include the following:

1. Hazard logging and monitoring, which trace hazards from preliminary hazard analysis through to testing and system validation.
2. Safety reviews, which are used throughout the development process.
3. Safety certification, where the safety of critical components is formally certified. This involves a group external to the system development team

examining the available evidence and deciding whether or not a system or component should be considered to be safe before it is made available for use.

To support these safety assurance processes, project safety engineers should be appointed who have explicit responsibility for the safety aspects of a system. This means that these individuals will be held responsible if a safety-related system failure occurs. They must be able to demonstrate that the safety assurance activities have been properly carried out.

Safety engineers work with quality managers to ensure that a detailed configuration management system is used to track all safety-related documentation and keep it in step with the associated technical documentation. This is essential in all dependable processes. There is little point in having stringent validation procedures if a failure of configuration management means that the wrong system is delivered to the customer. Configuration and quality management are covered in Chapters 24 and 25.

The hazard analysis process that is an essential part of safety-critical systems development is an example of a safety assurance process. Hazard analysis is concerned with identifying hazards, their probability of occurrence, and the probability of each hazard leading to an accident. If there is program code that checks for and handles each hazard, then you can argue that these hazards will not result in accidents. Such arguments may be supplemented by safety arguments, as discussed later in this chapter. Where external certification is required before a system is used (e.g., in an aircraft), it is usually a condition of certification that this traceability can be demonstrated.

The central safety document that should be produced is the hazard log. This document provides evidence of how identified hazards have been taken into account during software development. This hazard log is used at each stage of the software development process to document how that development stage has taken the hazards into account. A simplified example of a hazard log entry for the insulin delivery system is shown in Figure 15.6. This form documents the process of hazard analysis and shows design requirements that have been generated during this process. These design requirements are intended to ensure that the control system can never deliver an insulin overdose to a user of the insulin pump.

As shown in Figure 15.6, individuals who have safety responsibilities should be explicitly identified. This is important for two reasons:

1. When people are identified, they can be held accountable for their actions. This means that they are likely to take more care because any problems can be traced back to their work.

2. In the event of an accident, there may be legal proceedings or an enquiry. It is important to be able to identify who was responsible for safety assurance so that they can account for their actions.

Hazard Log					Page 4: Printed 20.02.2009
System: Insulin Pump System *Safety Engineer:* James Brown			*File:* InsulinPump/Safety/HazardLog *Log version:* 1/3		
Identified Hazard	Insulin overdose delivered to patient				
Identified by	Jane Williams				
Criticality class	1				
Identified risk	High				
Fault tree identified	YES	*Date*	24.01.07	*Location*	Hazard Log, Page 5
Fault tree creators	Jane Williams and Bill Smith				
Fault tree checked	YES	*Date*	28.01.07	*Checker*	James Brown

System safety design requirements

1. The system shall include self-testing software that will test the sensor system, the clock, and the insulin delivery system.
2. The self-checking software shall be executed once per minute.
3. In the event of the self-checking software discovering a fault in any of the system components, an audible warning shall be issued and the pump display shall indicate the name of the component where the fault has been discovered. The delivery of insulin shall be suspended.
4. The system shall incorporate an override system that allows the system user to modify the computed dose of insulin that is to be delivered by the system.
5. The amount of override shall be no greater than a pre-set value (maxOverride), which is set when the system is configured by medical staff.

Figure 15.6 A simplified hazard log entry

15.5 Safety and dependability cases

Security and dependability assurance processes generate a lot of information. This may include test results, information about the development processes used, records of review meetings, etc. This information provides evidence about the security and dependability of a system, and is used to help decide whether or not the system is dependable enough for operational use.

Safety and dependability cases are structured documents setting out detailed arguments and evidence that a system is safe or that a required level of security or dependability has been achieved. They are sometimes called assurance cases. Essentially, a safety or dependability case pulls together all of the available evidence that demonstrates that a system is trustworthy. For many types of critical system, the production of a safety case is a legal requirement. The case must satisfy a regulator or certification body before the system can be deployed.

The responsibility of a regulator is to check that a completed system is as safe or dependable as practicable, so their role primarily comes into play when a

development project is complete. However, regulators and developers rarely work in isolation; they communicate with the development team to establish what has to be included in the safety case. The regulator and developers jointly examine processes and procedures to make sure that these are being enacted and documented to the regulator's satisfaction.

Dependability cases are usually developed during and after the system development process. This can sometimes cause problems if the development process activities do not produce evidence for the system's dependability. Graydon et al. (2007) argue that the development of a safety and dependability case should be tightly integrated with system design and implementation. This means that system design decisions may be influenced by the requirements of the dependability case. Design choices that may add significantly to the difficulties and costs of case development can be avoided.

Dependability cases are generalizations of system safety cases. A safety case is a set of documents that includes a description of the system to be certified, information about the processes used to develop the system and, critically, logical arguments that demonstrate that the system is likely to be safe. More succinctly, Bishop and Bloomfield (1998) define a safety case as:

> *A documented body of evidence that provides a convincing and valid argument that a system is adequately safe for a given application in a given environment.*

The organization and contents of a safety or dependability case depend on the type of system that is to be certified and its context of operation. Figure 15.7 shows one possible structure for a safety case but there are no widely used industrial standards in this area for safety cases. Safety case structures vary, depending on the industry and the maturity of the domain. For example, nuclear safety cases have been required for many years. They are very comprehensive and presented in a way that is familiar to nuclear engineers. However, safety cases for medical devices have been introduced much more recently. Their structure is more flexible and the cases themselves are less detailed than nuclear cases.

Of course, software itself is not dangerous. It is only when it is embedded in a large computer-based or sociotechnical system that software failures can result in failures of other equipment or processes that can cause injury or death. Therefore, a software safety case is always part of a wider system safety case that demonstrates the safety of the overall system. When constructing a software safety case, you have to relate software failures to wider system failures and demonstrate either that these software failures will not occur or that they will not be propagated in such a way that dangerous system failures may occur.

15.5.1 Structured arguments

The decision on whether or not a system is sufficiently dependable to be used should be based on logical arguments. These should demonstrate that the evidence presented supports the claims about a system's security and dependability. These claims may be absolute (event X will or will not happen) or probabilistic (the probability of

Chapter	Description
System description	An overview of the system and a description of its critical components.
Safety requirements	The safety requirements abstracted from the system requirements specification. Details of other relevant system requirements may also be included.
Hazard and risk analysis	Documents describing the hazards and risks that have been identified and the measures taken to reduce risk. Hazard analyses and hazard logs.
Design analysis	A set of structured arguments (see Section 15.5.1) that justify why the design is safe.
Verification and validation	A description of the V & V procedures used and, where appropriate, the test plans for the system. Summaries of the test results showing defects that have been detected and corrected. If formal methods have been used, a formal system specification and any analyses of that specification. Records of static analyses of the source code.
Review reports	Records of all design and safety reviews.
Team competences	Evidence of the competence of all of the team involved in safety-related systems development and validation.
Process QA	Records of the quality assurance processes (see Chapter 24) carried out during system development.
Change management processes	Records of all changes proposed, actions taken and, where appropriate, justification of the safety of these changes. Information about configuration management procedures and configuration management logs.
Associated safety cases	References to other safety cases that may impact the safety case.

Figure 15.7 The contents of a software safety case

occurrence of event Y is 0.n). An argument links the evidence and the claim. As shown in Figure 15.8, an argument is a relationship between what is thought to be the case (the claim) and a body of evidence that has been collected. The argument, essentially, explains why the claim, which is an assertion about system security or dependability, can be inferred from the available evidence.

For example, the insulin pump is a safety-critical device whose failure could cause injury to a user. In many countries, this means that a regulatory authority (in the UK, the Medical Devices Directorate) has to be convinced of the system's safety before the device can be sold and used. To make this decision, the regulator assesses the safety case for the system, which presents structured arguments that normal operation of the system will not cause harm to a user.

Safety cases usually rely on structured, claim-based arguments. For example, the following argument might be used to justify a claim that computations carried out by the control software will not lead to an overdose of insulin being delivered to a pump user. Of course, this is a very simplified argument. In a real safety case more detailed references to the evidence would be presented.

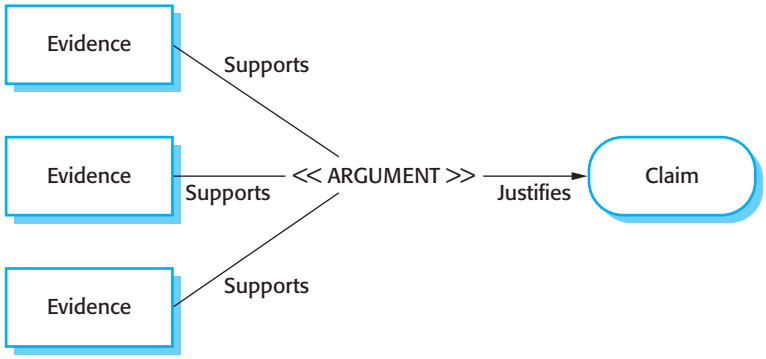


Figure 15.8 Structured arguments

Claim: The maximum single dose computed by the insulin pump will not exceed maxDose, where maxDose has been assessed as a safe single dose for a particular patient.

Evidence: Safety argument for insulin pump software control program (I discuss safety arguments later in this section).

Evidence: Test data sets for insulin pump. In 400 tests, the value of currentDose was correctly computed and never exceeded maxDose.

Evidence: Static analysis report for insulin pump control program. The static analysis of the control software revealed no anomalies that affected the value of currentDose, the program variable that holds the dose of insulin to be delivered.

Argument: The evidence presented shows that the maximum dose of insulin that can be computed is equal to maxDose.

It is therefore reasonable to assume, with a high level of confidence, that the evidence justifies the claim that the insulin pump will not compute a dose of insulin to be delivered that exceeds the maximum single dose.

Notice that the evidence presented is both redundant and diverse. The software is checked using several different mechanisms with significant overlap between them. As I discussed in Chapter 13, the use of redundant and diverse processes increases confidence. If there are omissions and mistakes that are not detected by one validation process, there is a good chance that these will be found by one of the others.

Of course, there will normally be many claims about the dependability and security of a system, with the validity of one claim often depending on whether or not other claims are valid. Therefore, claims may be organized in a hierarchy. Figure 15.9 shows part of this claim hierarchy for the insulin pump. To demonstrate that a high-level claim is valid, you first have to work through the arguments for lower-level claims. If you can show that each of these lower-level claims is justified, then you may be able to infer that the higher-level claims are justified.

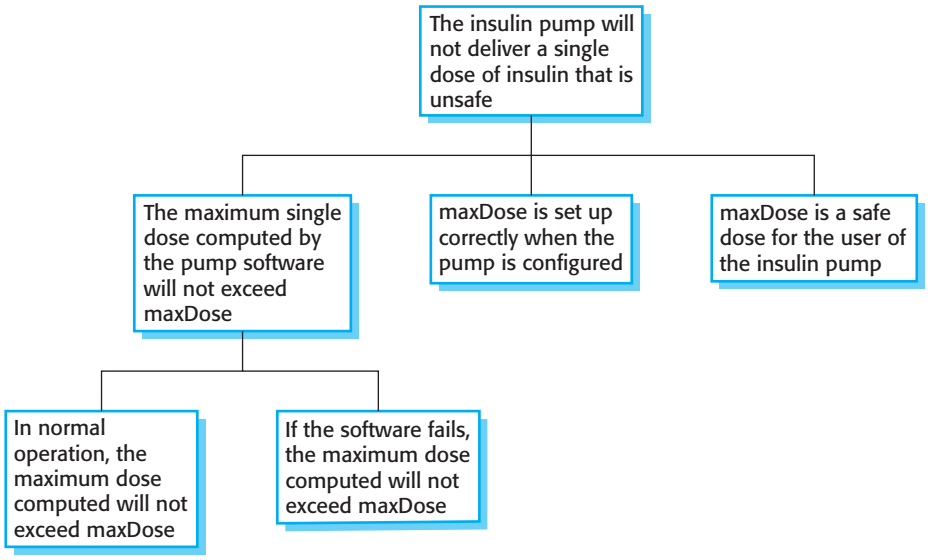


Figure 15.9 A safety claim hierarchy for the insulin pump

15.5.2 Structured safety arguments

Structured safety arguments are a type of structured argument, which demonstrate that a program meets its safety obligations. In a safety argument, it is not necessary to prove that the program works as intended. It is only necessary to show that program execution cannot result in an unsafe state. This means that safety arguments are cheaper to make than correctness arguments. You don't have to consider all program states—you can simply concentrate on states that could lead to an accident.

A general assumption that underlies work in system safety is that the number of system faults that can lead to safety-critical hazards is significantly less than the total number of faults that may exist in the system. Safety assurance can concentrate on these faults that have hazard potential. If it can be demonstrated that these faults cannot occur or, if they occur, the associated hazard will not result in an accident, then the system is safe. This is the basis of structured safety arguments.

Structured safety arguments are intended to demonstrate that, assuming normal execution conditions, a program should be safe. They are usually based on contradiction. The steps involved in creating a safety argument are the following:

1. You start by assuming that an unsafe state, which has been identified by the system hazard analysis, can be reached by executing the program.
2. You write a predicate (a logical expression) that defines this unsafe state.
3. You then systematically analyze a system model or the program and show that, for all program paths leading to that state, the terminating condition of these paths contradicts the unsafe state predicate. If this is the case, the initial assumption of an unsafe state is incorrect.

```
— The insulin dose to be delivered is a function of
— blood sugar level, the previous dose delivered and
— the time of delivery of the previous dose
currentDose = computeInsulin () ;

// Safety check-adjust currentDose if necessary.

// if statement 1
if (previousDose == 0)
{
   if (currentDose > maxDose/2)
      currentDose = maxDose/2 ;
}
else
   if (currentDose > (previousDose * 2) )
      currentDose = previousDose * 2 ;

// if statement 2

if ( currentDose < minimumDose )
   currentDose = 0 ;
else if ( currentDose > maxDose )
   currentDose = maxDose ;
administerInsulin (currentDose) ;
```

Figure 15.10 Insulin dose computation with safety checks

4. When you have repeated this analysis for all identified hazards then you have strong evidence that the system is safe.

Structured safety arguments can be applied at different levels, from requirements through design models to code. At the requirements level, you are trying to demonstrate that there are no missing safety requirements and that the requirements do not make invalid assumptions about the system. At the design level, you might analyze a state model of the system to find unsafe states. At the code level, you consider all of the paths through the safety-critical code to show that the execution of all paths leads to a contradiction.

As an example, consider the code in Figure 15.10, which might be part of the implementation of the insulin delivery system. The code computes the dose of insulin to be delivered then applies some safety checks to reduce the probability than an overdose of insulin will be injected. Developing a safety argument for this code involves demonstrating that the dose of insulin administered is never greater than the maximum safe level for a single dose. This is established for each individual diabetic user in discussions with their medical advisors.

To demonstrate safety, you do not have to prove that the system delivers the 'correct' dose, merely that it never delivers an overdose to the patient. You work on the assumption that maxDose is the safe level for that system user.

To construct the safety argument, you identify the predicate that defines the unsafe state, which is that currentDose > maxDose. You then demonstrate that all program paths lead to a contradiction of this unsafe assertion. If this is the case, the

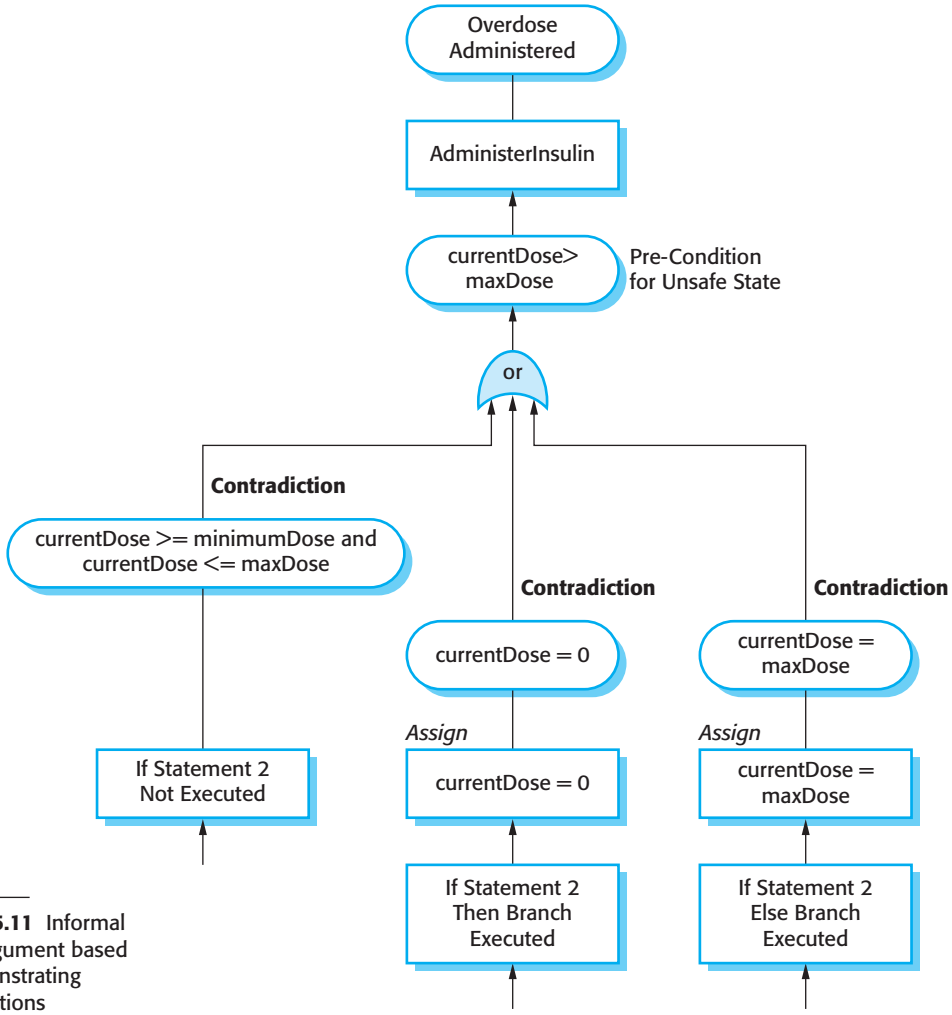


Figure 15.11 Informal safety argument based on demonstrating contradictions

unsafe condition cannot be true. If you can do this, you can be confident that the program will not compute an unsafe dose of insulin. You can structure and present the safety arguments graphically, as shown in Figure 15.11.

To construct a structured argument for a program does not make an unsafe computation, you first identify all possible paths through the code that could lead to the potentially unsafe state. You work backwards from this unsafe state and consider the last assignment to all of the state variables on each path leading to this unsafe state. If you can show that none of the values of these variables is unsafe, then you have shown that your initial assumption (that the computation is unsafe) is incorrect.

Working backwards is important because it means you can ignore all states apart from the final states that lead to the exit condition for the code. The previous values don't matter to the safety of the system. In this example, all you need to be concerned with is the set of possible values of currentDose immediately

before the administerInsulin method is executed. You can ignore computations, such as if-statement 1 in Figure 15.10, in the safety argument because their results are over-written in later program statements.

In the safety argument shown in Figure 15.11, there are three possible program paths that lead to the call to the administerInsulin method. You have to show that the amount of insulin delivered never exceeds maxDose. All possible program paths to administerInsulin are considered:

1. Neither branch of if-statement 2 is executed. This can only happen if currentDose is either greater than or equal to minimumDose and less than or equal to maxDose. This is the post-condition—an assertion that is true after the statement has been executed.
2. The then-branch of if-statement 2 is executed. In this case, the assignment setting currentDose to zero is executed. Therefore, its post-condition is currentDose = 0.
3. The else-if-branch of if-statement 2 is executed. In this case, the assignment setting currentDose to maxDose is executed. Therefore, after this statement has been executed, we know that the post-condition is currentDose = maxDose.

In all three cases, the post-conditions contradict the unsafe pre-condition that the dose administered is greater than maxDose. We can therefore claim that the computation is safe.

Structured arguments can be used in the same way to demonstrate that certain security properties of a system are true. For example, if you wish to show that a computation will never lead to the permissions on a resource being changed, you may be able to use a structured security argument to show this. However, the evidence from structured arguments is less reliable for security validation. This is because there is a possibility that the attacker may corrupt the code of the system. In such a case, the code executed is not the code that you have claimed is secure.

KEY POINTS

- Static analysis is an approach to V & V that examines the source code (or other representation) of a system, looking for errors and anomalies. It allows all parts of a program to be checked, not just those parts that are exercised by system tests.
- Model checking is a formal approach to static analysis that exhaustively checks all states in a system for potential errors.
- Statistical testing is used to estimate software reliability. It relies on testing the system with a test data set that reflects the operational profile of the software. Test data may be generated automatically.

- Security validation is difficult because security requirements state what should not happen in a system, rather than what should. Furthermore, system attackers are intelligent and may have more time to probe for weaknesses than is available for security testing.
- Security validation may be carried out using experience-based analysis, tool-based analysis, or 'tiger teams' that simulate attacks on a system.
- It is important to have a well-defined, certified process for safety-critical systems development. The process must include the identification and monitoring of potential hazards.
- Safety and dependability cases collect all of the evidence that demonstrates a system is safe and dependable. Safety cases are required when an external regulator must certify the system before it is used.
- Safety cases are usually based on structured arguments. Structured safety arguments show that an identified hazardous condition can never occur by considering all program paths that lead to an unsafe condition, and showing that the condition cannot hold.

FURTHER READING

Software Reliability Engineering: More Reliable Software, Faster and Cheaper, 2nd edition. This is probably the definitive book on the use of operational profiles and reliability models for reliability assessment. It includes details of experiences with statistical testing. (J. D. Musa, McGraw-Hill, 2004.)

'NASA's Mission Reliable'. A discussion of how NASA has used static analysis and model checking for assuring the reliability of spacecraft software. (P. Regan and S. Hamilton, *IEEE Computer*, **37** (1), January 2004.) http://dx.doi.org/10.1109/MC.2004.1260727.

Dependability cases. An example-based introduction to defining a dependability case. (C. B. Weinstock, J. B. Goodenough, J. J. Hudak, Software Engineering Institute, CMU/SEI-2004-TN-016, 2004.) http://www.sei.cmu.edu/publications/documents/04.reports/04tn016.html.

How to Break Web Software: Functional and Security Testing of Web Applications and Web Services. A short book that provides good practical advice on how to run security tests on networked applications. (M. Andrews and J. A. Whittaker, Addison-Wesley, 2006.)

'Using static analysis to find bugs'. This paper describes Findbugs, a Java static analyzer that uses simple techniques to find potential security violations and runtime errors. (N. Ayewah et al., *IEEE Software*, **25** (5), Sept/Oct 2008.) http://dx.doi.org/10.1109/MS.2008.130.

EXERCISES

15.1. Explain when it may be cost effective to use formal specification and verification in the development of safety-critical software systems. Why do you think that critical systems engineers are against the use of formal methods?

15.2. Suggest a list of conditions that could be detected by a static analyzer for Java, C++, or any another programming language that you use. Comment on this list compared to the list given in Figure 15.2.

15.3. Explain why it is practically impossible to validate reliability specifications when these are expressed in terms of a very small number of failures over the total lifetime of a system.

15.4. Explain why ensuring system reliability is not a guarantee of system safety.

15.5. Using examples, explain why security testing is a very difficult process.

15.6. Suggest how you would go about validating a password protection system for an application that you have developed. Explain the function of any tools that you think may be useful.

15.7. The MHC-PMS has to be secure against attacks that might reveal confidential patient information. Some of these attacks have been discussed in Chapter 14. Using this information, extend the checklist in Figure 15.5 to guide testers of the MHC-PMS.

15.8. List four types of systems that may require software safety cases, explaining why safety cases are required.

15.9. The door lock control mechanism in a nuclear waste storage facility is designed for safe operation. It ensures that entry to the storeroom is only permitted when radiation shields are in place or when the radiation level in the room falls below some given value (dangerLevel). So:

i. If remotely controlled radiation shields are in place within a room, an authorized operator may open the door.

ii. If the radiation level in a room is below a specified value, an authorized operator may open the door.

iii. An authorized operator is identified by the input of an authorized door entry code.

The code shown in Figure 15.12 (see below) controls the door-locking mechanism. Note that the safe state is that entry should not be permitted. Using the approach discussed in section 15.5.2, develop a safety argument for this code. Use the line numbers to refer to specific statements. If you find that the code is unsafe, suggest how it should be modified to make it safe.

```
1       entryCode = lock.getEntryCode () ;
2       if (entryCode == lock.authorizedCode)
3       {
4             shieldStatus = Shield.getStatus ();
5             radiationLevel = RadSensor.get ();
6             if (radiationLevel < dangerLevel)
7                   state = safe;
8             else
9                   state = unsafe;
10            if (shieldStatus == Shield.inPlace() )
11                  state = safe;
12            if (state == safe)
13                  {
14                        Door.locked = false ;
15                        Door.unlock ();
16                  }
17            else
18            {
19                  Door.lock ( );
20                  Door.locked := true ;
21            }
22      }
```

Figure 15.12 Door entry code

15.10. Assume you were part of a team that developed software for a chemical plant, which failed, causing a serious pollution incident. Your boss is interviewed on television and states that the validation process is comprehensive and that there are no faults in the software. She asserts that the problems must be due to poor operational procedures. A newspaper approaches you for your opinion. Discuss how you should handle such an interview.

REFERENCES

Abrial, J. R. (2005). *The B Book: Assigning Programs to Meanings*. Cambridge, UK: Cambridge University Press.

Anderson, R. (2001). *Security Engineering: A Guide to Building Dependable Distributed Systems*. Chichester, UK: John Wiley & Sons.

Baier, C. and Katoen, J.-P. (2008). *Principles of Model Checking*. Cambridge, Mass.: MIT Press.

Ball, T., Bounimova, E., Cook, B., Levin, V., Lichtenberg, J., McGarvey, C., Ondrusek, B., S. K., R. and Ustuner, A. (2006). 'Thorough Static Analysis of Device Drivers'. *Proc. EuroSys* 2006, Leuven, Belgium.

Bishop, P. and Bloomfield, R. E. (1998). 'A methodology for safety case development'. *Proc. Safety-critical Systems Symposium*, Birmingham, UK: Springer.

Chandra, S., Godefroid, P. and Palm, C. (2002). 'Software model checking in practice: An industrial case study'. *Proc. 24th Int. Conf. on Software Eng*. (ICSE 2002), Orland, Fla.: IEEE Computer Society, 431–41.

Croxford, M. and Sutton, J. (2006). 'Breaking Through the V and V Bottleneck'. *Proc. 2nd Int. Eurospace—Ada-Europe Symposium on Ada in Europe*, Frankfurt, Germany: Springer-LNCS, 344–54.

Evans, D. and Larochelle, D. (2002). 'Improving Security Using Extensible Lightweight Static Analysis'. *IEEE Software*, **19** (1), 42–51.

Graydon, P. J., Knight, J. C. and Strunk, E. A. (2007). 'Assurance Based Development of Critical Systems'. *Proc. 37th Annual IEEE Conf. on Dependable Systems and Networks*, Edinburgh, Scotland: 347–57.

Holzmann, G. J. (2003). *The SPIN Model Checker*. Boston: Addison-Wesley.

Knight, J. C. and Leveson, N. G. (2002). 'Should software engineers be licensed?' *Comm*. ACM, **45** (11), 87–90.

Larus, J. R., Ball, T., Das, M., Deline, R., Fahndrich, M., Pincus, J., Rajamani, S. K. and Venkatapathy, R. (2003). 'Righting Software'. *IEEE Software*, **21** (3), 92–100.

Musa, J. D. (1998). *Software Reliability Engineering: More Reliable Software, Faster Development and Testing*. New York: McGraw-Hill.

Nguyen, T. and Ourghanlian, A. (2003). 'Dependability assessment of safety-critical system software by static analysis methods'. *Proc. IEEE Conf. on Dependable Systems and Networks (DSN' 2003)*, San Francisco, Calif.: IEEE Computer Society, 75–9.

Pfleeger, C. P. and Pfleeger, S. L. (2007). *Security in Computing, 4th edition*. Boston: Addison-Wesley.

Prowell, S. J., Trammell, C. J., Linger, R. C. and Poore, J. H. (1999). *Cleanroom Software Engineering: Technology and Process*. Reading, Mass.: Addison-Wesley.

Regan, P. and Hamilton, S. (2004). 'NASA's Mission Reliable'. *IEEE Computer*, **37** (1), 59–68.

Schneider, S. (1999). *Concurrent and Real-time Systems: The CSP Approach*. Chichester, UK: John Wiley and Sons.

Visser, W., Havelund, K., Brat, G., Park, S. and Lerda, F. (2003). 'Model Checking Programs'. *Automated Software Engineering J*., **10** (2), 203–32.

Voas, J. (1997). 'Fault Injection for the Masses'. *IEEE Computer*, **30** (12), 129–30.

Wordsworth, J. (1996). *Software Engineering with* B. Wokingham: Addison-Wesley.

Zheng, J., Williams, L., Nagappan, N., Snipes, W., Hudepohl, J. P. and Vouk, M. A. (2006). 'On the value of static analysis for fault detection in software'. *IEEE Trans. on Software Eng*., **32** (4), 240–5.

This page intentionally left blank



PART 3 Advanced Software Engineering

I have entitled this part of the book 'Advanced Software Engineering' because you have to understand the basics of the discipline, covered in Chapters 1–9, to benefit from the material covered here. Many of the topics discussed here reflect industrial software engineering practice in the development of distributed and real-time systems.

Software reuse has now become the dominant development paradigm for web-based information systems and enterprise systems. The most common approach to reuse is COTS reuse where a large system is configured for the needs of an organization and little or no original software development is required. I introduce the general topic of reuse in Chapter 16 and focus in this chapter on the reuse of COTS systems.

Chapter 17 is also concerned with the reuse of software components rather than entire software systems. Component-based software engineering is a process of component composition, with new code being developed to integrate reusable components. In this chapter, I explain what is meant by a component and why standard component models are needed for effective component reuse. I also discuss the general process of component-based software engineering and the problems of component composition.

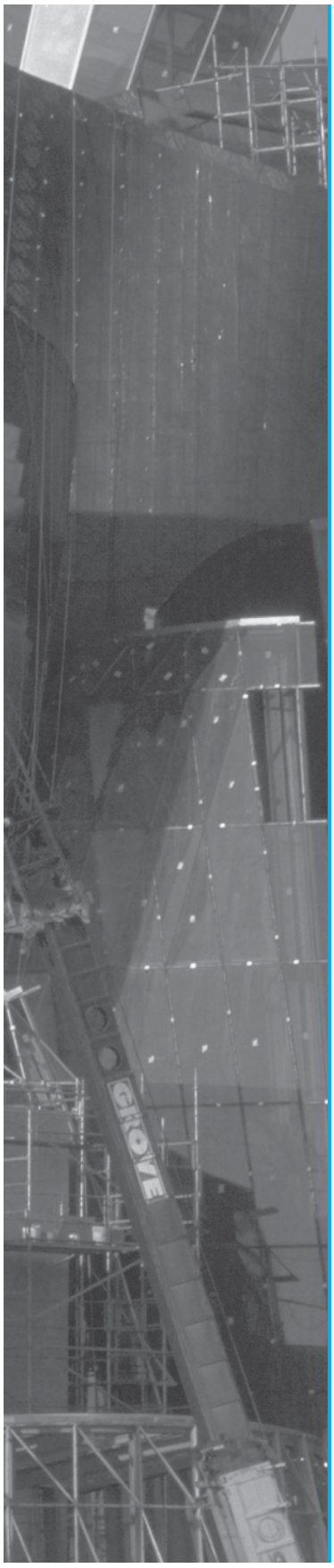
The majority of large systems are now distributed systems and Chapter 18 covers issues and problems of building distributed systems. I introduce the

client–server approach as a fundamental paradigm of distributed systems engineering, and explain various ways of implementing this architectural style. The final section in this chapter explains how providing software as a distributed application service will radically change the market for software products.

Chapter 19 introduces the related topic of service-oriented architectures, which link the notions of distribution and reuse. Services are reusable software components whose functionality can be accessed over the Internet and made available to a range of clients. In this chapter, I explain what is involved in creating services (service engineering) and composing services to create new software systems.

Embedded systems are the most widely used instances of software systems and Chapter 20 covers this important topic. I introduce the idea of a real-time embedded system and describe three architectural patterns that are used in embedded systems design. I then go on to explain the process of timing analysis and conclude the chapter with a discussion of real-time operating systems.

Finally, Chapter 21 covers aspect-oriented software development (AOSD). AOSD is also related to reuse and proposes a new approach, based on aspects, to organizing and structuring software systems. Although not yet mainstream software engineering, AOSD has the potential to significantly improve our current approaches to software implementation.



16

Software reuse

Objectives

The objectives of this chapter are to introduce software reuse and to describe approaches to system development based on large-scale system reuse. When you have read this chapter, you will:

- understand the benefits and problems of reusing software when developing new systems;
- understand the concept of an application framework as a set of reusable objects and how frameworks can be used in application development;
- have been introduced to software product lines, which are made up of a common core architecture and configurable, reusable components;
- have learned how systems can be developed by configuring and composing off-the-shelf application software systems.

Contents

16.1 The reuse landscape
16.2 Application frameworks
16.3 Software product lines
16.4 COTS product reuse

Reuse-based software engineering is a software engineering strategy where the development process is geared to reusing existing software. Although reuse was proposed as a development strategy more than 40 years ago (McIlroy, 1968), it is only since 2000 that 'development with reuse' has become the norm for new business systems. The move to reuse-based development has been in response to demands for lower software production and maintenance costs, faster delivery of systems, and increased software quality. More and more companies see their software as a valuable asset. They are promoting reuse to increase their return on software investments.

The availability of reusable software has increased dramatically. The open source movement has meant that there is a huge reusable code base available at low cost. This may be in the form of program libraries or entire applications. There are many domain-specific application systems available that can be tailored and adapted to the needs of a specific company. Some large companies provide a range of reusable components for their customers. Standards, such as web service standards, have made it easier to develop general services and reuse them across a range of applications.

Reuse-based software engineering is an approach to development that tries to maximize the reuse of existing software. The software units that are reused may be of radically different sizes. For example:

1. *Application system reuse* The whole of an application system may be reused by incorporating it without changing into other systems or by configuring the application for different customers. Alternatively, application families that have a common architecture, but which are tailored for specific customers, may be developed. I cover application system reuse later in this chapter.
2. *Component reuse* Components of an application, ranging in size from subsystems to single objects, may be reused. For example, a pattern-matching system developed as part of a text-processing system may be reused in a database management system. I cover component reuse in Chapters 17 and 19.
3. *Object and function reuse* Software components that implement a single function, such as a mathematical function, or an object class may be reused. This form of reuse, based around standard libraries, has been common for the past 40 years. Many libraries of functions and classes are freely available. You reuse the classes and functions in these libraries by linking them with newly developed application code. In areas such as mathematical algorithms and graphics, where specialized expertise is needed to develop efficient objects and functions, this is a particularly effective approach.

Software systems and components are potentially reusable entities, but their specific nature sometimes means that it is expensive to modify them for a new situation. A complementary form of reuse is 'concept reuse' where, rather than reuse a software component, you reuse an idea, a way, or working or an algorithm. The concept that you reuse is represented in an abstract notation (e.g., a system model), which does not include implementation detail. It can, therefore, be configured and adapted for a range of situations. Concept reuse can be embodied in approaches such as design

Benefit	Explanation
Increased dependability	Reused software, which has been tried and tested in working systems, should be more dependable than new software. Its design and implementation faults should have been found and fixed.
Reduced process risk	The cost of existing software is already known, whereas the costs of development are always a matter of judgment. This is an important factor for project management because it reduces the margin of error in project cost estimation. This is particularly true when relatively large software components such as subsystems are reused.
Effective use of specialists	Instead of doing the same work over and over again, application specialists can develop reusable software that encapsulates their knowledge.
Standards compliance	Some standards, such as user interface standards, can be implemented as a set of reusable components. For example, if menus in a user interface are implemented using reusable components, all applications present the same menu formats to users. The use of standard user interfaces improves dependability because users make fewer mistakes when presented with a familiar interface.
Accelerated development	Bringing a system to market as early as possible is often more important than overall development costs. Reusing software can speed up system production because both development and validation time may be reduced.

Figure 16.1 Benefits of software reuse

patterns (covered in Chapter 7), configurable system products, and program generators. When concepts are reused, the reuse process includes an activity where the abstract concepts are instantiated to create executable reusable components.

An obvious advantage of software reuse is that overall development costs should be reduced. Fewer software components need to be specified, designed, implemented, and validated. However, cost reduction is only one advantage of reuse. In Figure 16.1, I have listed other advantages of reusing software assets.

However, there are costs and problems associated with reuse (Figure 16.2). There is a significant cost associated with understanding whether or not a component is suitable for reuse in a particular situation, and in testing that component to ensure its dependability. These additional costs mean that the reductions in overall development costs through reuse may be less than anticipated.

As I discussed in Chapter 2, software development processes have to be adapted to take reuse into account. In particular, there has to be a requirements refinement stage where the requirements for the system are modified to reflect the reusable software that is available. The design and implementation stages of the system may also include explicit activities to look for and evaluate candidate components for reuse.

Software reuse is most effective when it is planned as part of an organization-wide reuse program. A reuse program involves the creation of reusable assets and the adaptation of development processes to incorporate these assets in new software. The importance of reuse planning has been recognized for many years in Japan (Matsumoto, 1984), where reuse is an integral part of the Japanese ‘factory’ approach

Problem	Explanation
Increased maintenance costs	If the source code of a reused software system or component is not available, then maintenance costs may be higher because the reused elements of the system may become increasingly incompatible with system changes.
Lack of tool support	Some software tools do not support development with reuse. It may be difficult or impossible to integrate these tools with a component library system. The software process assumed by these tools may not take reuse into account. This is particularly true for tools that support embedded systems engineering, less so for object-oriented development tools.
Not-invented-here syndrome	Some software engineers prefer to rewrite components because they believe they can improve on them. This is partly to do with trust and partly to do with the fact that writing original software is seen as more challenging than reusing other people's software.
Creating, maintaining, and using a component library	Populating a reusable component library and ensuring the software developers can use this library can be expensive. Development processes have to be adapted to ensure that the library is used.
Finding, understanding, and adapting reusable components	Software components have to be discovered in a library, understood and, sometimes, adapted to work in a new environment. Engineers must be reasonably confident of finding a component in the library before they include a component search as part of their normal development process.

Figure 16.2 Problems with reuse

to software development (Cusamano, 1989). Companies such as Hewlett-Packard have also been very successful in their reuse programs (Griss and Wosser, 1995), and their experience has been documented in a book by Jacobson et al. (1997).

16.1 The reuse landscape

Over the past 20 years, many techniques have been developed to support software reuse. These techniques exploit the facts that systems in the same application domain are similar and have potential for reuse; that reuse is possible at different levels from simple functions to complete applications; and that standards for reusable components facilitate reuse. Figure 16.3 sets out a number of possible ways of implementing software reuse, with each described briefly in Figure 16.4.

Given this array of techniques for reuse, the key question is "which is the most appropriate technique to use in a particular situation?" Obviously, this depends on the requirements for the system being developed, the technology and reusable assets available, and the expertise of the development team. Key factors that you should consider when planning reuse are:

1. *The development schedule for the software* If the software has to be developed quickly, you should try to reuse off-the-shelf systems rather than individual components. These are large-grain reusable assets. Although the fit to

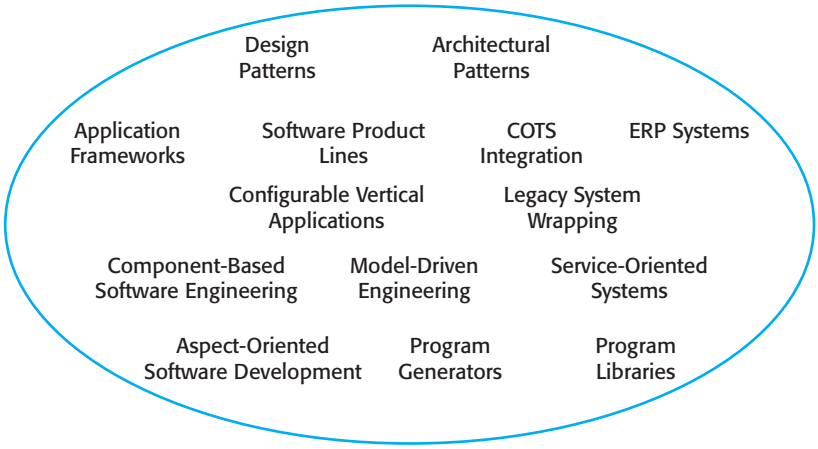


Figure 16.3 The reuse landscape

requirements may be imperfect, this approach minimizes the amount of development required.

2. *The expected software lifetime* If you are developing a long-lifetime system, you should focus on the maintainability of the system. You should not just think about the immediate benefits of reuse but also of the long-term implications.

   Over its lifetime, you will have to adapt the system to new requirements, which will mean making changes to parts of the system. If you do not have access to the source code, you may prefer to avoid off-the-shelf components and systems from external suppliers; suppliers may not be able to continue support for the reused software.

3. *The background, skills, and experience of the development team* All reuse technologies are fairly complex and you need quite a lot of time to understand and use them effectively. Therefore, if the development team has skills in a particular area, this is probably where you should focus.

4. *The criticality of the software and its non-functional requirements* For a critical system that has to be certified by an external regulator, you may have to create a dependability case for the system (discussed in Chapter 15). This is difficult if you don't have access to the source code of the software. If your software has stringent performance requirements, it may be impossible to use strategies such as generator-based reuse, where you generate the code from a reusable domain-specific representation of a system. These systems often generate relatively inefficient code.

5. *The application domain* In some application domains, such as manufacturing and medical information systems, there are several generic products that may be reused by configuring them to a local situation. If you are working in such a domain, you should always consider these as an option.

Approach	Description
Architectural patterns	Standard software architectures that support common types of application systems are used as the basis of applications. Described in Chapters 6, 13, and Chapter 20.
Design patterns	Generic abstractions that occur across applications are represented as design patterns showing abstract and concrete objects and interactions. Described in Chapter 7.
Component-based development	Systems are developed by integrating components (collections of objects) that conform to component-model standards. Described in Chapter 17.
Application frameworks	Collections of abstract and concrete classes are adapted and extended to create application systems.
Legacy system wrapping	Legacy systems (see Chapter 9) are 'wrapped' by defining a set of interfaces and providing access to these legacy systems through these interfaces.
Service-oriented systems	Systems are developed by linking shared services, which may be externally provided. Described in Chapter 19.
Software product lines	An application type is generalized around a common architecture so that it can be adapted for different customers.
COTS product reuse	Systems are developed by configuring and integrating existing application systems.
ERP systems	Large-scale systems that encapsulate generic business functionality and rules are configured for an organization.
Configurable vertical applications	Generic systems are designed so that they can be configured to the needs of specific system customers.
Program libraries	Class and function libraries that implement commonly used abstractions are available for reuse.
Model-driven engineering	Software is represented as domain models and implementation independent models and code is generated from these models. Described in Chapter 5.
Program generators	A generator system embeds knowledge of a type of application and is used to generate systems in that domain from a user-supplied system model.
Aspect-oriented software development	Shared components are woven into an application at different places when the program is compiled. Described in Chapter 21.

Figure 16.4 Approaches that support software reuse

6. *The platform on which the system will run* Some components models, such as .NET, are specific to Microsoft platforms. Similarly, generic application systems may be platform-specific and you may only be able to reuse these if your system is designed for the same platform.



Generator-based reuse

Generator-based reuse involves incorporating reusable concepts and knowledge into automated tools and providing an easy way for tool users to integrate specific code with this generic knowledge. This approach is usually most effective in domain-specific applications. Known solutions to problems in that domain are embedded in the generator system and selected by the user to create a new system.

http://www.SoftwareEngineering-9.com/Web/Reuse/Generator.html

The range of available reuse techniques is such that, in most situations, there is the possibility of some software reuse. Whether or not reuse is achieved is often a managerial rather than a technical issue. Managers may be unwilling to compromise their requirements to allow reusable components to be used. They may not understand the risks associated with reuse as well as they understand the risks of original development. Although the risks of new software development may be higher, some managers may prefer known to unknown risks.

16.2 Application frameworks

Early enthusiasts for object-oriented development suggested that one of the key benefits of using an object-oriented approach was that objects could be reused in different systems. However, experience has shown that objects are often too small and are specialized for a particular application. It takes longer to understand and adapt the object than to reimplement it. It has now become clear that object-oriented reuse is best supported in an object-oriented development process through larger-grain abstractions called frameworks.

As the name suggests, a framework is a generic structure that is extended to create a more specific subsystem or application. Schmidt et al. (2004) define a framework to be:

> *". . . an integrated set of software artefacts (such as classes, objects and components) that collaborate to provide a reusable architecture for a family of related applications."*

Frameworks provide support for generic features that are likely to be used in all applications of a similar type. For example, a user interface framework will provide support for interface event handling and will include a set of widgets that can be used to construct displays. It is then left to the developer to specialize these by adding specific functionality for a particular application. For example, in a user interface framework, the developer defines display layouts that are appropriate to the application being implemented.

Frameworks support design reuse in that they provide a skeleton architecture for the application as well as the reuse of specific classes in the system. The architecture

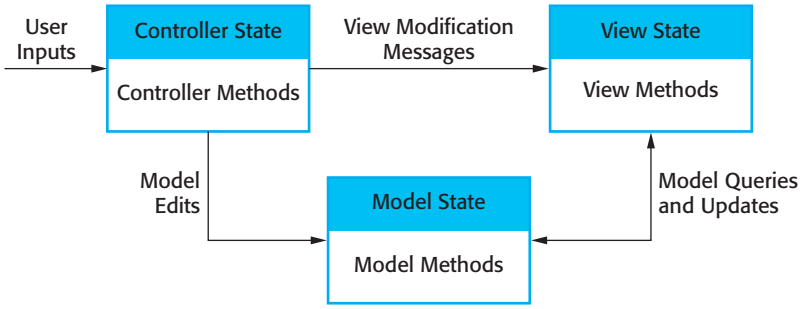


Figure 16.5 The Model-View-Controller pattern

is defined by the object classes and their interactions. Classes are reused directly and may be extended using features such as inheritance.

Frameworks are implemented as a collection of concrete and abstract object classes in an object-oriented programming language. Therefore, frameworks are language-specific. There are frameworks available in all of the commonly used object-oriented programming languages (e.g., Java, C#, C++, as well as dynamic languages such as Ruby and Python). In fact, a framework can incorporate several other frameworks, where each of these is designed to support the development of part of the application. You can use a framework to create a complete application or to implement part of an application, such as the graphical user interface.

Fayad and Schmidt (1997) discuss three classes of frameworks:

1. *System infrastructure frameworks* These frameworks support the development of system infrastructures such as communications, user interfaces, and compilers (Schmidt, 1997).
2. *Middleware integration frameworks* These consist of a set of standards and associated object classes that support component communication and information exchange. Examples of this type of framework include Microsoft's .NET and Enterprise Java Beans (EJB). These frameworks provide support for standardized component models, as discussed in Chapter 17.
3. *Enterprise application frameworks* These are concerned with specific application domains such as telecommunications or financial systems (Baumer, et al., 1997). These embed application domain knowledge and support the development of end-user applications.

Web application frameworks (WAFs) are a more recent and very important type of framework. WAFs that support the construction of dynamic websites are now widely available. The architecture of a WAF is usually based on the Model-View-Controller (MVC) composite pattern (Gamma et al., 1995), shown in Figure 16.5.

The MVC pattern was originally proposed in the 1980s as an approach to GUI design that allowed for multiple presentations of an object and separate styles of interaction with each of these presentations. It allows for the separation of the application

state from the user interface to application. An MVC framework supports the presentation of data in different ways and allows interaction with each of these presentations. When the data is modified through one of the presentations, the system model is changed and the controllers associated with each view update their presentation.

Frameworks are often implementations of design patterns, as discussed in Chapter 7. For example, an MVC framework includes the Observer pattern, the Strategy pattern, the Composite pattern, and a number of others that are discussed by Gamma et al. (1995). The general nature of patterns and their use of abstract and concrete classes allows for extensibility. Without patterns, frameworks would, almost certainly, be impractical.

Web application frameworks usually incorporate one or more specialized frameworks that support specific application features. Although each framework includes slightly different functionality, most web application frameworks support the following features:

1. *Security* WAFs may include classes to help implement user authentication (login) and access control to ensure that users can only access permitted functionality in the system.
2. *Dynamic web pages* Classes are provided to help you define web page templates and to populate these dynamically with specific data from the system database.
3. *Database support* Frameworks don't usually include a database but rather assume that a separate database, such as MySQL, will be used. The framework may provide classes that provide an abstract interface to different databases.
4. *Session management* Classes to create and manage sessions (a number of interactions with the system by a user) are usually part of a WAF.
5. *User interaction* Most web frameworks now provide AJAX support (Holdener, 2008), which allows more interactive web pages to be created.

To extend a framework you do not change the framework code. Rather, you add concrete classes that inherit operations from abstract classes in the framework. In addition, you may have to define callbacks. Callbacks are methods that are called in response to events recognized by the framework. Schmidt et al. (2004) call this 'inversion of control'. The framework objects, rather than the application-specific objects, are responsible for control in the system. In response to events from the user interface, database, etc., these framework objects invoke 'hook methods' that are then linked to user-provided functionality. The application-specific functionality responds to the event in an appropriate way (Figure 16.6). For example, a framework will have a method that handles a mouse click from the environment. This method calls the hook method, which you must configure to call the appropriate application methods to handle the mouse click.

Applications that are constructed using frameworks can be the basis for further reuse through the concept of software product lines or application families. Because these applications are constructed using a framework, modifying family members to

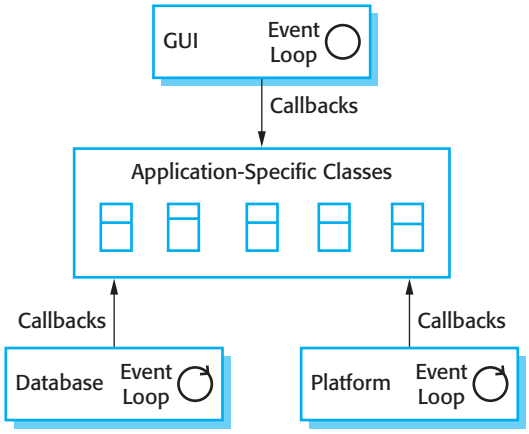


Figure 16.6 Inversion of control in frameworks

create instances of the system is often a straightforward process. It involves rewriting concrete classes and methods that you have added to the framework.

However, frameworks are usually more general than software product lines, which focus on a specific family of application system. For example, you can use a web-based framework to build different types of web-based applications. One of these might be a software product line that supports web-based help desks. This 'help desk product line' may then be further specialized to provide particular types of help desk support.

Frameworks are an effective approach to reuse, but are expensive to introduce into software development processes. They are inherently complex and it can take several months to learn to use them. It can be difficult and expensive to evaluate available frameworks to choose the most appropriate one. Debugging framework-based applications is difficult because you may not understand how the framework methods interact. This is a general problem with reusable software. Debugging tools may provide information about the reused system components, which a developer does not understand.

16.3 Software product lines

One of the most effective approaches to reuse is to create software product lines or application families. A software product line is a set of applications with a common architecture and shared components, with each application specialized to reflect different requirements. The core system is designed to be configured and adapted to suit the needs of different system customers. This may involve the configuration of some components, implementing additional components, and modifying some of the components to reflect new requirements.

Developing applications by adapting a generic version of the application means that a high proportion of the application code is reused. Furthermore, application experience is often transferable from one system to another. Consequently, when software engineers join a development team, their learning process is shortened. Testing is simplified because tests for large parts of the application may also be reused, thus reducing the overall application development time.

Software product lines usually emerge from existing applications. That is, an organization develops an application then, when a similar system is required, informally reuses code from this in the new application. The same process is used as other similar applications are developed. However, change tends to corrupt application structure so, as more new instances are developed, it becomes increasingly difficult to create a new version. Consequently, a decision to design a generic product line may then be made. This involves identifying common functionality in product instances and including this in a base application, which is then used for future development. This base application is deliberately structured to simplify reuse and reconfiguration.

Application frameworks and software product lines obviously have much in common. They both support a common architecture and components, and require new development to create a specific version of a system. The main differences between these approaches are as follows:

1. Application frameworks rely on object-oriented features such as inheritance and polymorphism to implement extensions to the framework. Generally, the framework code is not modified and the possible modifications are limited to whatever is allowed by the framework. Software product lines are not necessarily created using an object-oriented approach. Application components are changed, deleted, or rewritten. There are no limits, in principle at least, to the changes that can be made.
2. Application frameworks are primarily focused on providing technical rather than domain-specific support. For example, there are application frameworks to create web-based applications. A software product line usually embeds detailed domain and platform information. For example, there could be a software product line concerned with web-based applications for health record management.
3. Software product lines are often control applications for equipment. For example, there may be a software product line for a family of printers. This means that the product line has to provide support for hardware interfacing. Application frameworks are usually software-oriented and they rarely provide support for hardware interfacing.
4. Software product lines are made up of a family of related applications, owned by the same organization. When you create a new application, your starting point is often the closest member of the application family, not the generic core application.

If you are developing a software product line using an object-oriented programming language, then you may use an application framework as a basis for the system. You create the core of the product line by extending the framework with domain-specific

components using its built-in mechanisms. There is then a second phase of development where versions of the system for different customers are created.

Various types of specialization of a software product line may be developed:

1. *Platform specialization* Versions of the application are developed for different platforms. For example, versions of the application may exist for Windows, Mac OS, and Linux platforms. In this case, the functionality of the application is normally unchanged; only those components that interface with the hardware and operating system are modified.
2. *Environment specialization* Versions of the application are created to handle particular operating environments and peripheral devices. For example, a system for the emergency services may exist in different versions, depending on the vehicle communications system. In this case, the system components are changed to reflect the functionality of the communications equipment used.
3. *Functional specialization* Versions of the application are created for specific customers who have different requirements. For example, a library automation system may be modified depending on whether it is used in a public library, a reference library, or a university library. In this case, components that implement functionality may be modified and new components added to the system.
4. *Process specialization* The system is adapted to cope with specific business processes. For example, an ordering system may be adapted to cope with a centralized ordering process in one company and a distributed process in another.

The architecture of a software product line often reflects a general, application-specific architectural style or pattern. For example, consider a product line system that is designed to handle vehicle despatching for emergency services. Operators of this system take calls about incidents, find the appropriate vehicle to respond to the incident and dispatch the vehicle to the incident site. The developers of such a system may market versions of this for police, fire, and ambulance services.

This vehicle despatching system is an example of a resource management architecture (Figure 16.7). You can see how this four-layer structure is instantiated in Figure 16.8, which shows the modules that might be included in a vehicle despatching system product line. The components at each level in the product line system are as follows:

1. At the interaction level, there are components providing an operator display interface and an interface with the communications systems used.
2. At the I/O management level (level 2), there are components that handle operator authentication, generate reports of incidents and vehicles despatched, support map output and route planning, and provide a mechanism for operators to query the system databases.

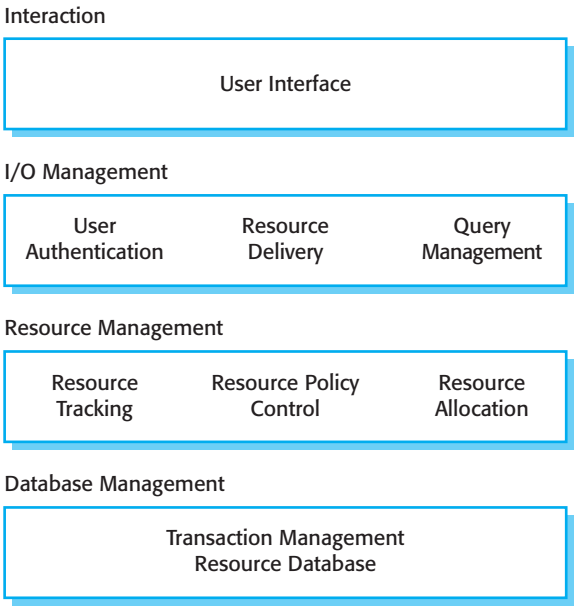


Figure 16.7 The architecture of a resource allocation system

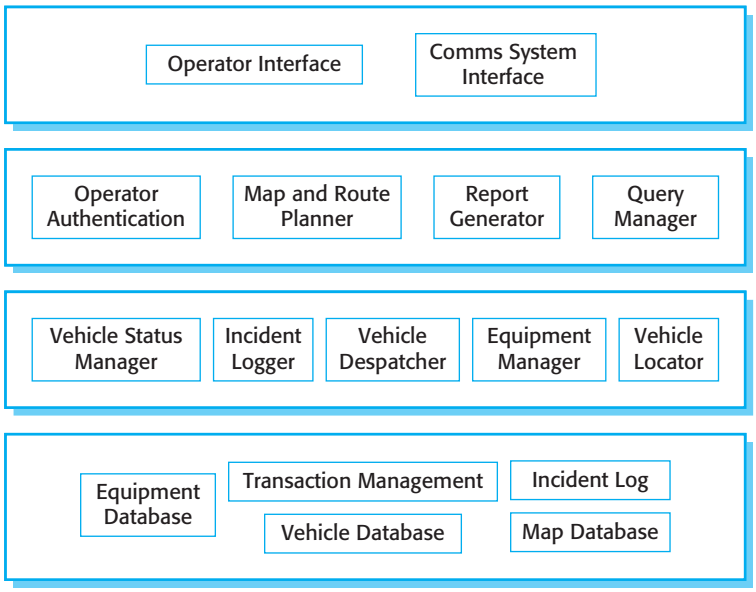


Figure 16.8 The product line architecture of a vehicle dispatcher system

3. At the resource management level (level 3) there are components that allow vehicles to be located and despatched, components to update the status of vehicles and equipment, and a component to log details of incidents.
4. At the database level, as well as the usual transaction management support, there are separate databases of vehicles, equipment, and maps.

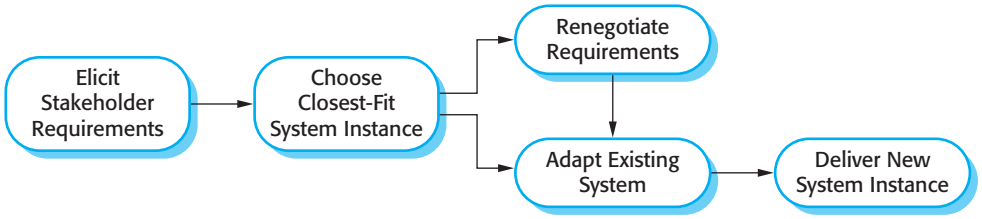


Figure 16.9 Product instance development

To create a specific version of this system, you may have to modify individual components. For example, the police have a large number of vehicles but a small number of vehicle types, whereas the fire service has many types of specialized vehicles. Therefore, you may have to define a different vehicle database structure when implementing a system for these different services.

Figure 16.9 shows the steps involved in extending a software product line to create a new application. The steps involved in this general process are as follows:

1. *Elicit stakeholder requirements* You may start with a normal requirements engineering process. However, because a system already exists, you will need to demonstrate the system and have stakeholders experiment with it, expressing their requirements as modifications to the functions provided.
2. *Select the existing system that is the closest fit to the requirements* When creating a new member of a product line, you may start with the nearest product instance. The requirements are analyzed and the family member that is the closest fit is chosen for modification.
3. *Renegotiate requirements* As more details of required changes emerge and the project is planned, there may be some requirements renegotiation to minimize the changes that are needed.
4. *Adapt existing system* New modules are developed for the existing system and existing system modules are adapted to meet the new requirements.
5. *Deliver new family member* The new instance of the product line is delivered to the customer. At this stage, you should document its key features so that it may be used as a basis for other system developments in the future.

When you create a new member of product line you may have to find a compromise between reusing as much of the generic application as possible and satisfying detailed stakeholder requirements. The more detailed the system requirements, the less likely it is that the existing components will meet these requirements. However, if stakeholders are willing to be flexible and to limit the system modifications that are required, you can usually deliver the system more quickly and at a lower cost.

Software product lines are designed to be reconfigured and this reconfiguration may involve adding or removing components from the system, defining parameters and constraints for system components, and including knowledge of business

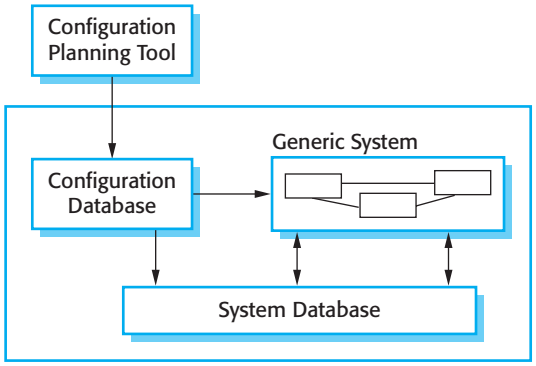


Figure 16.10 Deployment-time configuration

processes. This configuration may occur at different stages in the development process:

1. *Design-time configuration* The organization that is developing the software modifies a common product line core by developing, selecting, or adapting components to create a new system for a customer.
2. *Deployment-time configuration* A generic system is designed for configuration by a customer or consultants working with the customer. Knowledge of the customer's specific requirements and the system's operating environment is embedded in a set of configuration files that are used by the generic system.

When a system is configured at design time, the supplier starts with either a generic system or an existing product instance. By modifying and extending modules in this system, they create a specific system that delivers the required customer functionality. This usually involves changing and extending the source code of the system so greater flexibility is possible than with deployment-time configuration.

Deployment-time configuration involves using a configuration tool to create a specific system configuration that is recorded in a configuration database or as a set of configuration files (Figure 16.10). The executing system consults this database when executing so that its functionality may be specialized to its execution context.

There are several levels of deployment-time configuration that may be provided in a system:

1. Component selection, where you select the modules in a system that provide the required functionality. For example, in a patient information system, you may select an image management component that allows you to link medical images (x-rays, CT scans, etc.) to the patient's medical record.
2. Workflow and rule definition, where you define workflows (how information is processed, stage by stage) and validation rules that should apply to information entered by users or generated by the system.

3. Parameter definition, where you specify the values of specific system parameters that reflect the instance of the application that you are creating. For example, you may specify the maximum length of fields for data input by a user or the characteristics of hardware attached to the system.

Deployment-time configuration can be very complex and it may take many months to configure the system for a customer. Large configurable systems may support the configuration process by providing software tools, such as a configuration planning tools, to support the configuration process. I discuss deployment-time configuration further in Section 16.4.1. This covers the reuse of COTS systems that have to be configured to work in different operational environments.

Design-time configuration is used when it is impossible to use the existing deployment-time configuration facilities in a system to develop a new system version. However, over time, when you have created several family members with comparable functionality, you may decide to refactor the core product line to include functionality that has been implemented in several application family members. You then make that new functionality configurable when the system is deployed.

16.4 COTS product reuse

A commercial-off-the-shelf (COTS) product is a software system that can be adapted to the needs of different customers without changing the source code of the system. Virtually all desktop software and a wide variety of server products are COTS software. Because this software is designed for general use, it usually includes many features and functions. It therefore has the potential to be reused in different environments and as part of different applications. Torchiano and Morisio (2004) also discovered that using open source products were often used as COTS products. That is, the open source systems were used without change and without looking at the source code.

COTS products are adapted by using built-in configuration mechanisms that allow the functionality of the system to be tailored to specific customer needs. For example, in a hospital patient record system, separate input forms and output reports might be defined for different types of patient. Other configuration features may allow the system to accept plug-ins that extend functionality or check user inputs to ensure that they are valid.

This approach to software reuse has been very widely adopted by large companies over the last 15 or so years, as it offers significant benefits over customized software development:

1. As with other types of reuse, more rapid deployment of a reliable system may be possible.

2. It is possible to see what functionality is provided by the applications and so it is easier to judge whether or not they are likely to be suitable. Other companies may already use the applications so experience of the systems is available.
3. Some development risks are avoided by using existing software. However, this approach has its own risks, as I discuss below.
4. Businesses can focus on their core activity without having to devote a lot of resources to IT systems development.
5. As operating platforms evolve, technology updates may be simplified as these are the responsibility of the COTS product vendor rather than the customer.

Of course, this approach to software engineering has its own problems:

1. Requirements usually have to be adapted to reflect the functionality and mode of operation of the COTS product. This can lead to disruptive changes to existing business processes.
2. The COTS product may be based on assumptions that are practically impossible to change. The customer must therefore adapt their business to reflect these assumptions.
3. Choosing the right COTS system for an enterprise can be a difficult process, especially as many COTS products are not well documented. Making the wrong choice could be disastrous as it may be impossible to make the new system work as required.
4. There may be a lack of local expertise to support systems development. Consequently, the customer has to rely on the vendor and external consultants for development advice. This advice may be biased and geared to selling products and services, rather than meeting the real needs of the customer.
5. The COTS product vendor controls system support and evolution. They may go out of business, be taken over, or may make changes that cause difficulties for customers.

Software reuse based on COTS has become increasingly common. The vast majority of new business information processing systems are now built using COTS rather than using an object-oriented approach. Although there are often problems with this approach to system development (Tracz, 2001), success stories (Baker, 2002; Balk and Kedia, 2000; Brownsword and Morris, 2003; Pfarr and Reis, 2002) show that COTS-based reuse reduces effort and the time to deploy the system.

There are two types of COTS product reuse, namely COTS-solution systems and COTS-integrated systems. COTS-solution systems consist of a generic application from a single vendor that is configured to customer requirements. COTS-integrated systems involve integrating two or more COTS systems (perhaps from different

COTS-solution systems	COTS-integrated systems
Single product that provides the functionality required by a customer	Several heterogeneous system products are integrated to provide customized functionality
Based around a generic solution and standardized processes	Flexible solutions may be developed for customer processes
Development focus is on system configuration	Development focus is on system integration
System vendor is responsible for maintenance	System owner is responsible for maintenance
System vendor provides the platform for the system	System owner provides the platform for the system

Figure 16.11 COTS-solution and COTS-integrated systems

vendors) to create an application system. Figure 16.11 summarizes the differences between these different approaches.

16.4.1 COTS-solution systems

COTS-solution systems are generic application systems that may be designed to support a particular business type, business activity, or sometimes, a complete business enterprise. For example, a COTS-solution system may be produced for dentists that handles appointments, dental records, patient recall, etc. At a larger scale, an Enterprise Resource Planning (ERP) system may support all of the manufacturing, ordering, and customer relationship management activities in a large company.

Domain-specific COTS-solution systems, such as systems to support a business function (e.g., document management), provide functionality that is likely to be required by a range of potential users. However, they also incorporate built-in assumptions about how users work and these may cause problems in specific situations. For example, a system to support student registration in a university may assume that students will be registered for one degree at one university. However, if universities collaborate to offer joint degrees, then it may be practically impossible to represent this in the system.

ERP systems, such as those produced by SAP and BEA, are large-scale integrated systems designed to support business practices such as ordering and invoicing, inventory management, and manufacturing scheduling (O'Leary, 2000). The configuration process for these systems involves gathering detailed information about the customer's business and business processes, and embedding this in a configuration database. This often requires detailed knowledge of configuration notations and tools and is usually carried out by consultants working alongside system customers.

A generic ERP system includes a number of modules that may be composed in different ways to create a system for a customer. The configuration process

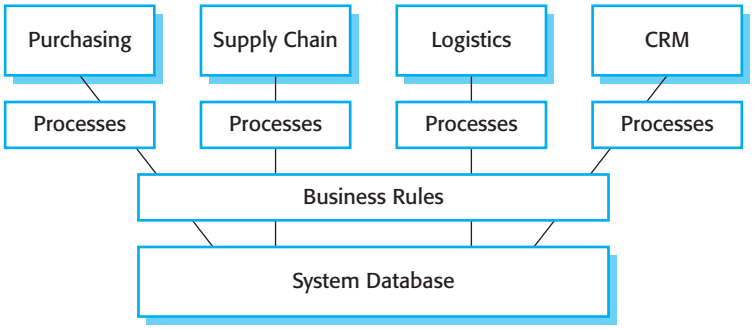


Figure 16.12 The architecture of an ERP system

involves choosing which modules are to be included, configuring these individual modules, defining business processes and business rules, and defining the structure and organization of the system database. A model of the overall architecture of an ERP system that supports a range of business functions is shown in Figure 16.12.

The key features of this architecture are:

1. A number of modules to support different business functions. These are large-grain modules that may support entire departments or divisions of the business. In the example shown in Figure 16.12, the modules that have been selected for inclusion in the system are a module to support purchasing, a module to support supply chain management, a logistics module to support the delivery of goods, and a customer relationship management (CRM) module to maintain customer information.
2. A defined set of business processes, associated with each module, which relate to activities in that module. For example, there may be a definition of the ordering process that defines how orders are created and approved. This will specify the roles and activities involved in placing an order.
3. A common database that maintains information about all related business functions. This means that it should not be necessary to replicate information, such as customer details, in different parts of the business.
4. A set of business rules that apply to all data in the database. Therefore, when data is input from one function, these rules should ensure that it is consistent with the data required by other functions. For example, there may be a business rule that all expense claims have to be approved by someone more senior than the person making the claim.

ERP systems are used in almost all large companies to support some or all of their functions. They are, therefore, a very widely used form of software reuse. However, the obvious limitation of this approach to reuse is that the functionality of the system

is restricted to the functionality of the generic core. Furthermore, a company's processes and operations have to be expressed in the system configuration language, and there may be a mismatch between the concepts in the business and the concepts supported in the configuration language.

For example, in an ERP system that was sold to a university, the concept of a customer had to be defined. This caused great difficulties when configuring the system. However, universities have multiple types of customers, such as students, research funding agencies, educational charities, etc., each of which have different characteristics. None of them are really comparable to the notion of a commercial customer (i.e., a person or business that buys products or services). A serious mismatch between the business model used by the system and that of the buyer of the system makes it highly probable that the ERP system will not meet the buyer's real needs (Scott, 1999).

Both domain-specific COTS products and ERP systems usually require extensive configuration to adapt them to the requirements of each organization where they are installed. This configuration may involve:

1. Selecting the required functionality from the system (e.g., by deciding what modules should be included).
2. Establishing a data model that defines how the organization's data will be structured in the system database.
3. Defining business rules that apply to that data.
4. Defining the expected interactions with external systems.
5. Designing the input forms and the output reports generated by the system.
6. Designing new business processes that conform to the underlying process model supported by the system.
7. Setting parameters that define how the system is deployed on its underlying platform.

Once the configuration settings are completed, a COTS-solution system is then ready for testing. Testing is a major problem when systems are configured rather than programmed using a conventional language. Because these systems are built using a reliable platform, obvious system failures and crashes are relatively rare. Rather the problems are often subtle and relate to the interactions between the operational processes and the system configuration. These may only be detectable by end-users and so may not be discovered during the system testing process. Furthermore, automated unit testing, supported by testing frameworks such as JUnit, cannot be used. The underlying system is unlikely to support any kind of test automation and there may be no complete system specification that can be used to derive system tests.

16.4.2 COTS-integrated systems

COTS-integrated systems are applications that include two or more COTS products or, sometimes, legacy application systems. You may use this approach when there is no single COTS system that meets all of your needs or when you wish to integrate a new COTS product with systems that you already use. The COTS products may interact through their APIs (Application Programming Interfaces) or service interfaces if these are defined. Alternatively, they may be composed by connecting the output of one system to the input of another or by updating the databases used by the COTS applications.

To develop systems using COTS products, you have to make a number of design choices:

1. *Which COTS products offer the most appropriate functionality?* Typically, there will be several COTS products available, which can be combined in different ways. If you don't already have experience with a COTS product, it can be difficult to decide which product is the most suitable.
2. *How will data be exchanged?* Different products normally use unique data structures and formats. You have to write adaptors that convert from one representation to another. These adaptors are run-time systems that operate alongside the COTS products.
3. *What features of a product will actually be used?* COTS products may include more functionality than you need and functionality may be duplicated across different products. You have to decide which features in what product are most appropriate for your requirements. If possible, you should also deny access to unused functionality because this can interfere with normal system operation. The failure of the first flight of the Ariane 5 rocket (Nuseibeh, 1997) was a consequence of a failure in an inertial navigation system that was reused from the Ariane 4 system. However, the functionality that failed was not actually required in Ariane 5.

Consider the following scenario as an illustration of COTS integration. A large organization intends to develop a procurement system that allows staff to place orders from their desk. By introducing this system across the organization, the company estimates that it can save $5 million per year. By centralizing buying, the new procurement system can ensure that orders are always made from suppliers who offer the best prices and should reduce the paperwork costs associated with orders. As with manual systems, the system involves choosing the goods available from a supplier, creating an order, having the order approved, sending the order to a supplier, receiving the goods, and confirming that payment should be made.

The company has a legacy ordering system that is used by a central procurement office. This order processing software is integrated with an existing invoicing and delivery system. To create the new ordering system, the legacy system is integrated with a web-based e-commerce platform and an e-mail system that

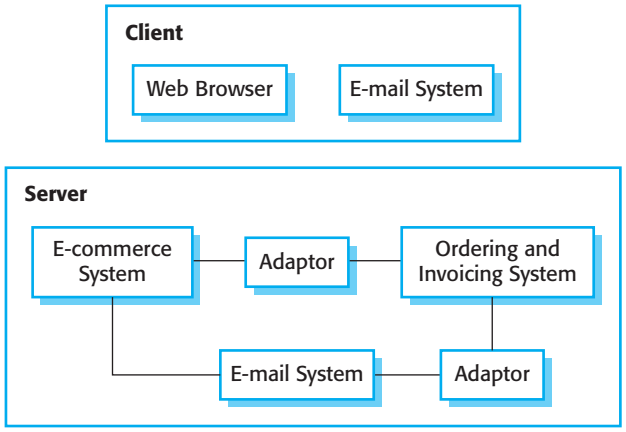


Figure 16.13 A COTS-integrated procurement system

handles communications with users. The structure of the final procurement system, constructed using COTS, is shown in Figure 16.13.

This procurement system is a client–server based and, on the client, standard web browsing and e-mail software are used. On the server, the e-commerce platform has to integrate with the existing ordering system through an adaptor. The e-commerce system has its own format for orders, confirmations of delivery, and so forth, and these have to be converted into the format used by the ordering system. The e-commerce system uses the e-mail system to send notifications to users, but the ordering system was never designed for this. Therefore, another adaptor has to be written to convert the notifications from the ordering system into e-mail messages.

Months, sometimes years, of implementation effort can be saved, and the time to develop and deploy a system can be drastically reduced using a COTS-integrated approach. The procurement system described above was implemented and deployed in a very large company in nine months, rather than the three years that they estimated would be required to develop the system in Java.

COTS integration can be simplified if a service-oriented approach is used. Essentially, a service-oriented approach means allowing access to the application system's functionality through a standard service interface, with a service for each discrete unit of functionality. Some applications may offer a service interface but, sometimes, this service interface has to be implemented by the system integrator. Essentially, you have to program a wrapper that hides the application and provides externally visible services (Figure 16.14). This approach is particularly valuable for legacy systems that have to be integrated with newer application systems.

In principle, integrating COTS products is the same as integrating any other components. You have to understand the system interfaces and use them exclusively to communicate with the software; you have to trade off specific requirements against rapid development and reuse; and you have to design a system architecture that allows the COTS systems to operate together.

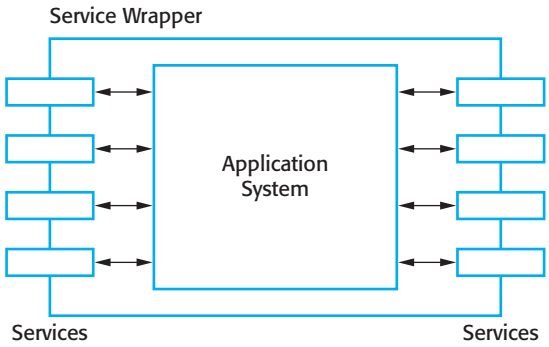


Figure 16.14
Application wrapping

However, the fact that these products are usually large systems in their own right, and are often sold as separate standalone systems, introduces additional problems. Boehm and Abts (1999) discuss four important COTS system integration problems:

1. *Lack of control over functionality and performance* Although the published interface of a product may appear to offer the required facilities, these may not be properly implemented or may perform poorly. The product may have hidden operations that interfere with its use in a specific situation. Fixing these problems may be a priority for the COTS product integrator but may not be of real concern for the product vendor. Users may simply have to find work-arounds to problems if they wish to reuse the COTS product.

2. *Problems with COTS system interoperability* It is sometimes difficult to get COTS products to work together because each product embeds its own assumptions about how it will be used. Garlan et al. (1995), reporting on their experience of trying to integrate four COTS products, found that three of these products were event-based but each used a different model of events. Each system assumed that it had exclusive access to the event queue. As a consequence, integration was very difficult. The project required five times as much effort as originally predicted. The schedule was extended to two years rather than the predicted six months. In a retrospective analysis of their work 10 years later, Garlan et al. (2009) concluded that the integration problems that they discovered had not been solved. Torchiano and Morisio (2004) found that lack of compliance with standards in some COTS products meant that integration was more difficult than anticipated.

3. *No control over system evolution* Vendors of COTS products make their own decisions on system changes, in response to market pressures. For PC products, in particular, new versions are often produced frequently and may not be compatible with all previous versions. New versions may have additional unwanted functionality, and previous versions may become unavailable and unsupported.

4. *Support from COTS vendors* The level of support available from COTS vendors varies widely. Vendor support is particularly important when problems arise as

developers do not have access to the source code and detailed documentation of the system. Although vendors may commit to providing support, changing market and economic circumstances may make it difficult for them to deliver this commitment. For example, a COTS system vendor may decide to discontinue a product because of limited demand, or they may be taken over by another company that does not wish to support all of the products that have been acquired.

Boehm and Abts reckon that, in many cases, the cost of system maintenance and evolution may be greater for COTS-integrated systems. All of the above difficulties are life-cycle problems; they don't just affect the initial development of the system. The further removed the people involved in the system maintenance become from the original system developers, the more likely it is that real difficulties will arise with the integrated COTS products.

KEY POINTS

- Most new business software systems are now developed by reusing knowledge and code from previously implemented systems.
- There are many different ways to reuse software. These range from the reuse of classes and methods in libraries to the reuse of complete application systems.
- The advantages of software reuse are lower costs, faster software development, and lower risks. System dependability is increased. Specialists can be used more effectively by concentrating their expertise on the design of reusable components.
- Application frameworks are collections of concrete and abstract objects that are designed for reuse through specialization and the addition of new objects. They usually incorporate good design practice through design patterns.
- Software product lines are related applications that are developed from one or more base applications. A generic system is adapted and specialized to meet specific requirements for functionality, target platform, or operational configuration.
- COTS product reuse is concerned with the reuse of large-scale, off-the-shelf systems. These provide a lot of functionality and their reuse can radically reduce costs and development time. Systems may be developed by configuring a single, generic COTS product or by integrating two or more COTS products.
- Enterprise Resource Planning systems are examples of large-scale COTS reuse. You create an instance of an ERP system by configuring a generic system with information about the customer's business processes and rules.
- Potential problems with COTS-based reuse include lack of control over functionality and performance, lack of control over system evolution, the need for support from external vendors, and difficulties in ensuring that systems can interoperate.

FURTHER READING

Reuse-based Software Engineering. A comprehensive discussion of different approaches to software reuse. The authors cover technical reuse issues and managing reuse processes. (H. Mili, A. Mili, S. Yacoub and E. Addy, John Wiley & Sons, 2002.)

'Overlooked Aspects of COTS-Based Development'. An interesting article that discusses a survey of developers using a COTS-based approach, and the problems that they encountered. (M. Torchiano and M. Morisio, *IEEE Software*, **21** (2), March–April 2004.) http://dx.doi.org/10.1109/MS.2004.1270770.

'Construction by Configuration: A New Challenge for Software Engineering'. This is an invited paper that I wrote in which I discuss the problems and difficulties of constructing a new application by configuring existing systems. (I. Sommerville, *Proc. 19th Australian Software Engineering Conference*, 2008.) http://dx.doi.org/10.1109/ASWEC.2008.75.

'Architectural Mismatch: Why Reuse Is Still So Hard'. This article looks back on an earlier paper that discussed the problems of reusing and integrating a number of COTS systems. The authors concluded that, although some progress has been made, there were still problems in conflicting assumptions made by the designers of the individual systems. (D. Garlan et al., *IEEE Software*, **26** (4), July–August 2009.) http://dx.doi.org//10.1109/MS.2009.86.

EXERCISES

16.1. What are the major technical and nontechnical factors that hinder software reuse? Do you personally reuse much software and, if not, why not?

16.2. Suggest why the savings in cost from reusing existing software are not simply proportional to the size of the components that are reused.

16.3. Give four circumstances where you might recommend against software reuse.

16.4. Explain what is meant by 'inversion of control' in application frameworks. Explain why this approach could cause problems if you integrated two separate systems that were originally created using the same application framework.

16.5. Using the example of the weather station system described in Chapters 1 and Chapter 7, suggest a product line architecture for a family of applications that are concerned with remote monitoring and data collection. You should present your architecture as a layered model, showing the components that might be included at each level.

16.6. Most desktop software, such as word processing software, can be configured in a number of different ways. Examine software that you regularly use and list the configuration options for that software. Suggest difficulties that users might have in configuring the software. If you use Microsoft Office or Open Office, these are good examples to use for this exercise.

16.7. Why have many large companies chosen ERP systems as the basis for their organizational information system? What problems may arise when deploying a large-scale ERP system in an organization?

16.8. Identify six possible risks that can arise when systems are constructed using COTS. What steps can a company take to reduce these risks?

16.9. Explain why adaptors are usually needed when systems are constructed by integrating COTS products. Suggest three practical problems that might arise in writing adaptor software to link two COTS application products.

16.10. The reuse of software raises a number of copyright and intellectual property issues. If a customer pays a software contractor to develop a system, who has the right to reuse the developed code? Does the software contractor have the right to use that code as a basis for a generic component? What payment mechanisms might be used to reimburse providers of reusable components? Discuss these issues and other ethical issues associated with the reuse of software.

REFERENCES

Baker, T. (2002). 'Lessons Learned Integrating COTS into Systems'. *Proc. ICCBSS 2002 (1st Int. Conf on COTS-based Software Systems*), Orlando, Fla:: Springer, 21–30.

Balk, L. D. and Kedia, A. (2000). 'PPT: A COTS Integration Case Study'. *Proc. Int. Conf. on Software Eng*., Limerick, Ireland: ACM Press, 42–9.

Baumer, D., Gryczan, G., Knoll, R., Lilienthal, C., Riehle, D. and Zullighoven, H. (1997). 'Framework Development for Large Systems'. *Comm*. ACM, **40** (10), 52–9.

Boehm, B. and Abts, C. (1999). 'COTS Integration: Plug and Pray?' *IEEE Computer*, **32** (1), 135–38.

Brownsword, L. and Morris, E. (2003). 'The Good News about COTS'. http://www.sei.cmu.edu/news-at-sei/features/2003/1q03/feature-1-1q03.htm

Cusamano, M. (1989). 'The Software Factory: A Historical Interpretation'. *IEEE Software*, **6** (2), 23–30.

Fayad, M. E. and Schmidt, D. C. (1997). 'Object-oriented Application Frameworks'. *Comm*. ACM, **40** (10), 32–38.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, Mass.: Addison-Wesley.

Garlan, D., Allen, R. and Ockerbloom, J. (1995). 'Architectural Mismatch: Why Reuse is so Hard'. *IEEE Software*, **12** (6), 17–26.

Garlan, D., Allen, R. and Ockerbloom, J. (2009). 'Architectural Mismatch: Why Reuse is Still so Hard'. *IEEE Software*, **26** (4), 66–9.

Griss, M. L. and Wosser, M. (1995). 'Making reuse work at Hewlett-Packard'. *IEEE Software*, **12** (1), 105–7.

Holdener, A. T. (2008). *Ajax: The Definitive Guide*. Sebastopol, Calif.: O'Reilly and Associates.

Jacobson, I., Griss, M. and Jonsson, P. (1997). *Software Reuse*. Reading, Mass.: Addison-Wesley.

Matsumoto, Y. (1984). 'Some Experience in Promoting Reusable Software: Presentation in Higher Abstract Levels'. *IEEE. Trans. on Software Engineering*, **SE-10** (5), 502–12.

McIlroy, M. D. (1968). 'Mass-produced software components'. *Proc. NATO Conf. on Software Eng*., Garmisch, Germany: Springer-Verlag.

Nuseibeh, B. (1997). 'Ariane 5: Who Dunnit?' *IEEE Software*, **14** (3), 15–6.

O'Leary, D. E. (2000). *Enterprise Resource Planning Systems: Systems, Life Cycle, Electronic Commerce and Risk*. Cambridge, UK: Cambridge University Press.

Pfarr, T. and Reis, J. E. (2002). 'The Integration of COTS/GOTS within NASA's HST Command and Control System'. *Proc. ICCBSS 2002 (1st Int. Conf on COTS-based Software Systems)*, Orlando, Fla.: Springer, 209–21.

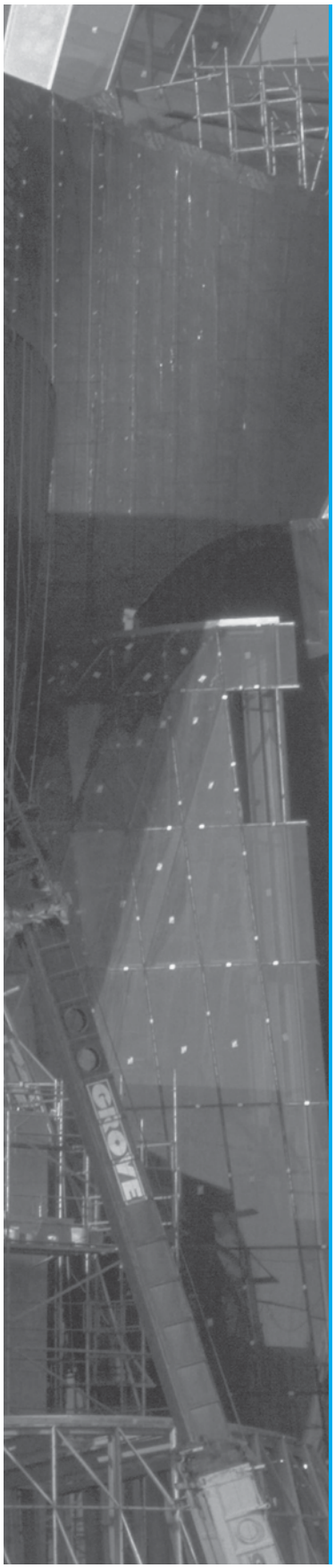
Schmidt, D. C. (1997). 'Applying design patterns and frameworks to develop object-oriented communications software'. In *Handbook of Programming Languages, Vol. 1.* (ed.). New York: Macmillan Computer Publishing.

Schmidt, D. C., Gokhale, A. and Natarajan, B. (2004). 'Leveraging Application Frameworks'. *ACM Queue*, **2** (5 (July/August)), 66–75.

Scott, J. E. (1999). 'The FoxMeyer Drug's Bankruptcy: Was it a Failure of ERP'. Proc. *Association for Information Systems 5th Americas Conf. on Information Systems*, Milwaukee, WI.

Torchiano, M. and Morisio, M. (2004). 'Overlooked Aspects of COTS-Based Development'. *IEEE Software*, **21** (2), 88–93.

Tracz, W. (2001). 'COTS Myths and Other Lessons Learned in *Component-Based Software Development*'. In *Component-based Software Engineering*. Heineman, G. T. and Councill, W. T. (ed.). Boston: Addison-Wesley, 99–112.



17

Component-based software engineering

Objectives

The objective of this chapter is to describe an approach to software reuse based on the composition of reusable, standardized components. When you have read this chapter you will:

- know that component-based software engineering is concerned with developing standardized components based on a component model, and composing these into application systems;
- understand what is meant by a component and a component model;
- know the principal activities in the CBSE process for reuse and the CBSE process with reuse;
- understand some of the difficulties and problems that arise during the process of component composition.

Contents

17.1 Components and component models
17.2 CBSE processes
17.3 Component composition

As I explained in Chapter 16, many new business systems are now developed by configuring off-the-shelf systems. However, when a company cannot use an off-the-shelf system because it does not meet their requirements, the software they need has to be specially developed. For custom software, component-based software engineering is an effective, reuse-oriented way to develop new enterprise systems.

Component-based software engineering (CBSE) emerged in the late 1990s as an approach to software systems development based on reusing software components. Its creation was motivated by designers' frustration that object-oriented development had not led to extensive reuse, as had been originally suggested. Single object classes were too detailed and specific, and often had to be bound with an application at compile time. You had to have detailed knowledge of the classes to use them, and this usually meant that you had to have the component source code. This meant that selling or distributing objects as individual reusable components was practically impossible.

Components are higher-level abstractions than objects and are defined by their interfaces. They are usually larger than individual objects and all implementation details are hidden from other components. CBSE is the process of defining, implementing, and integrating or composing loosely coupled, independent components into systems. It has become as an important software development approach because software systems are becoming larger and more complex. Customers are demanding more dependable software that is delivered and deployed more quickly. The only way that we can cope with complexity and deliver better software more quickly is to reuse rather than reimplement software components.

The essentials of component-based software engineering are:

1. Independent components that are completely specified by their interfaces. There should be a clear separation between the component interface and its implementation. This means that one implementation of a component can be replaced by another, without changing other parts of the system.
2. Component standards that facilitate the integration of components. These standards are embodied in a component model. They define, at the very minimum, how component interfaces should be specified and how components communicate. Some models go much further and define interfaces that should be implemented by all conformant components. If components conform to standards, then their operation is independent of their programming language. Components written in different languages can be integrated into the same system.
3. Middleware that provides software support for component integration. To make independent, distributed components work together, you need middleware support that handles component communications. Middleware for component support handles low-level issues efficiently and allows you to focus on application-related problems. In addition, middleware for component support may provide support for resource allocation, transaction management, security, and concurrency.
4. A development process that is geared to component-based software engineering. You need a development process that allows requirements to evolve, depending



Problems with CBSE

CBSE is now a mainstream approach to software engineering—it is a good way to build systems. However, when used as an approach to reuse, problems include component trustworthiness, component certification, requirements compromises, and predicting the properties of components, especially when they are integrated with other components.

http://www.SoftwareEngineering-9.com/Web/CBSE/problems.html

on the functionality of available components. I discuss CBSE development processes in Section 17.2.

Component-based development embodies good software engineering practice. It makes sense to design a system using components, even if you have to develop rather than reuse these components. Underlying CBSE are sound design principles that support the construction of understandable and maintainable software:

1. Components are independent so they do not interfere with each other's operation. Implementation details are hidden. The component's implementation can be changed without affecting the rest of the system.
2. Components communicate through well-defined interfaces. If these interfaces are maintained, one component can be replaced by another, which provides additional or enhanced functionality.
3. Component infrastructures offer a range of standard services that can be used in application systems. This reduces the amount of new code that has to be developed.

The initial motivation for CBSE was the need to support both reuse and distributed software engineering. A component was seen as an element of a software system that could be accessed, using a remote procedure call mechanism, by other components running on separate computers. Each system that reused a component had to incorporate its own copy of that component. This idea of a component extended the notion of distributed objects, as defined in distributed systems models such as the CORBA specification (Pope, 1997). Several different protocols and standards have been developed to support this view of a component, such as Sun's Enterprise Java Beans (EJB), Microsoft's COM and .NET, and CORBA's CCM (Lau and Wang, 2007).

In practice, these multiple standards have hindered the uptake of CBSE. It was impossible for components developed using different approaches to work together. Components that are developed for different platforms, such as .NET or J2EE, cannot interoperate. Furthermore, the standards and protocols proposed were complex and difficult to understand. This was also a barrier to their adoption.

In response to these problems, the notion of a component as a service was developed, and standards were proposed to support service-oriented software engineering.

The most significant difference between a component as a service and the original notion of a component is that services are stand-alone entities that are external to a program using them. When you build a service-oriented system, you reference the external service rather than including a copy of that service in your system.

Service-oriented software engineering, which I discuss in Chapter 19, is therefore a type of component-based software engineering. It uses a simpler notion of a component than that originally proposed in CBSE. It has been driven, from the outset, by standards. In situations where COTS-based reuse is impractical, service-oriented CBSE is becoming the dominant approach for the development of business systems.

17.1 Components and component models

There is general agreement in the CBSE community that a component is an independent software unit that can be composed with other components to create a software system. Beyond that, however, people have proposed varying definitions of a software component. Councill and Heineman (2001) define a component as:

> *"A software element that conforms to a standard component model and can be independently deployed and composed without modification according to a composition standard."*

This definition is essentially based on standards so that a software unit that conforms to these standards is a component. Szyperski (2002), however, does not mention standards in his definition of a component but focuses instead on the key characteristics of components:

> *"A software component is a unit of composition with contractually-specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties."*

Both of these definitions are based on the notion of a component as an element that is included in a system, rather than a service that is referenced by the system. However, they are also compatible with the idea of a service as a component.

Szyperski also states that a component has no externally observable state. This means that copies of components are indistinguishable. However, some component models, such as the Enterprise Java Beans model, allow stateful components, so these do not correspond with Szyperski's definition. Although stateless components are certainly simpler to use, there are some systems where stateful components are more convenient and reduce system complexity.

What the above definitions have in common is that they agree that components are independent, and that they are the fundamental unit of composition in a system. In my view, a better definition of a component can be derived by combining these

Component Characteristic	Description
Standardized	Component standardization means that a component used in a CBSE process has to conform to a standard component model. This model may define component interfaces, component metadata, documentation, composition, and deployment.
Independent	A component should be independent—it should be possible to compose and deploy it without having to use other specific components. In situations where the component needs externally provided services, these should be explicitly set out in a 'requires' interface specification.
Composable	For a component to be composable, all external interactions must take place through publicly defined interfaces. In addition, it must provide external access to information about itself, such as its methods and attributes.
Deployable	To be deployable, a component has to be self-contained. It must be able to operate as a stand-alone entity on a component platform that provides an implementation of the component model. This usually means that the component is binary and does not have to be compiled before it is deployed. If a component is implemented as a service, it does not have to be deployed by a user of a component. Rather, it is deployed by the service provider.
Documented	Components have to be fully documented so that potential users can decide whether or not the components meet their needs. The syntax and, ideally, the semantics of all component interfaces should be specified.

Figure 17.1 Component characteristics

proposals. Figure 17.1 shows what I consider to be the essential characteristics of a component as used in CBSE.

A useful way of thinking about a component is as a provider of one or more services. When a system needs a service, it calls on a component to provide that service without caring about where that component is executing or the programming language used to develop the component. For example, a component in a library system might provide a search service that allows users to search different library catalogs. A component that converts from one graphical format to another (e.g., TIFF to JPEG) provides a data conversion service, etc.

Viewing a component as a service provider emphasizes two critical characteristics of a reusable component:

1. The component is an independent executable entity that is defined by its interfaces. You don't need any knowledge of its source code to use it. It can either be referenced as an external service or included directly in a program.
2. The services offered by a component are made available through an interface and all interactions are through that interface. The component interface is expressed in terms of parameterized operations and its internal state is never exposed.



Component and objects

Components are often implemented in object-oriented languages and, in some cases, accessing the 'provides' interface of a component is done through method calls. However, components and object classes are not the same thing. Unlike object classes, components are independently deployable, do not define types, are language-independent, and are based on a standard component model.

http://www.SoftwareEngineering-9.com/Web/CBSE/objects.html

Components have two related interfaces, as shown in Figure 17.2. These interfaces reflect the services that the component provides and the services that the component requires to operate correctly:

- The 'provides' interface defines the services provided by the component. This interface, essentially, is the component API. It defines the methods that can be called by a user of the component. In a UML component diagram, the 'provides' interface for a component is indicated by a circle at the end of a line from the component icon.
- The 'requires' interface specifies what services must be provided by other components in the system if a component is to operate correctly. If these are not available, then the component will not work. This does not compromise the independence or deployability of a component because the 'requires' interface does not define how these services should be provided. In the UML, the symbol for a 'requires' interface is a semicircle at the end of a line from the component icon. Notice that 'provides' and 'requires' interface icons can fit together like a ball and socket.

To illustrate these interfaces, Figure 17.3 shows a model of a component that has been designed to collect and collate information from an array of sensors. It runs autonomously to collect data over a period of time and, on request, provides collated data to a calling component. The 'provides' interface includes methods to add, remove, start, stop, and test sensors. The report method returns the sensor data that has been collected, and the listAll method provides information about the attached sensors. Although I have not shown this here, these methods have associated parameters specifying the sensor identifiers, locations, and so on.

The 'requires' interface is used to connect the component to the sensors. It assumes that sensors have a data interface, accessed through sensorData, and a management

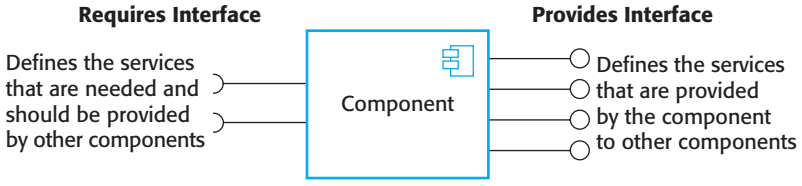


Figure 17.2 Component interfaces

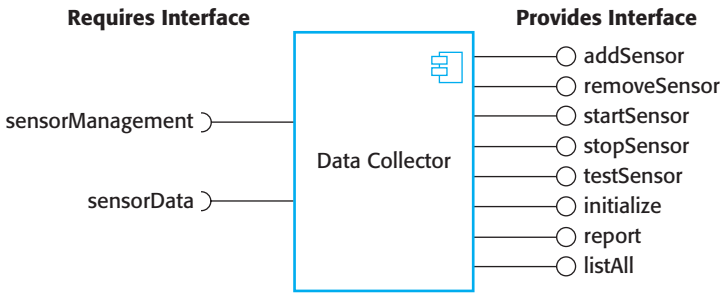


Figure 17.3 A model of a data collector component

interface, accessed through sensorManagement. This interface has been designed to connect to different types of sensor so it does not include specific sensor operations such as Test, provideReading, etc. Instead, the commands used by a specific type of sensor are embedded in a string, which is a parameter to the operations in the 'requires' interface. Adaptor components parse this string and translate the embedded commands into the specific control interface of each type of sensor. I discuss the use of adaptors later in this chapter, where I show how the data collector component is linked to a sensor (Figure 17.12).

A critical difference between a component as an external service and a component as a program element is that services are completely independent entities. They do not have a 'requires' interface. Different programs can use these services without the need to implement any additional support required by the service.

17.1.1 Component models

A component model is a definition of standards for component implementation, documentation, and deployment. These standards are for component developers to ensure that components can interoperate. They are also for providers of component execution infrastructures who provide middleware to support component operation. Many component models have been proposed, but the most important models are now the WebServices model, Sun's Enterprise Java Beans (EJB) model, and Microsoft's .NET model (Lau and Wang, 2007).

The basic elements of an ideal component model are discussed by Weinreich and Sametinger (2001). I summarize these model elements in Figure 17.4. This diagram shows that the elements of a component model define the component interfaces, the information that you need to use the component in a program, and how a component should be deployed:

1. *Interfaces* Components are defined by specifying their interfaces. The component model specifies how the interfaces should be defined and the elements, such as operation names, parameters, and exceptions, which should be included in the interface definition. The model should also specify the language used to define the component interfaces. For web services, this is WSDL, which I discuss in

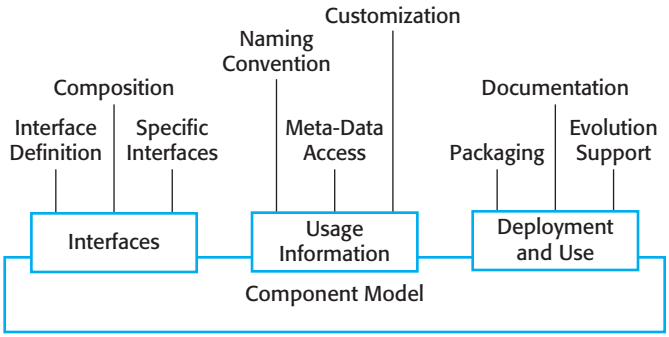


Figure 17.4 Basic elements of a component model

Chapter 19; EJB is Java-specific so Java is used as the interface definition language; in .NET, interfaces are defined using the Common Intermediate Language (CIL). Some component models require specific interfaces that must be defined by a component. These are used to compose the component with the component model infrastructure, which provides standardized services such as security and transaction management.

2. *Usage* In order for components to be distributed and accessed remotely, they need to have a unique name or handle associated with them. This has to be globally unique—for example, in EJB, a hierarchical name is generated with the root based on an Internet domain name. Services have a unique URI (Uniform Resource Identifier).

   Component meta-data is data about the component itself, such as information about its interfaces and attributes. The meta-data is important because it allows users of the component to find out what services are provided and required. Component model implementations normally include specific ways (such as the use of a reflection interface in Java) to access this component meta-data.

   Components are generic entities and, when deployed, they have to be configured to fit into an application system. For example, you could configure the Data collector component (Figure 17.2) by defining the maximum number of sensors in a sensor array. The component model may therefore specify how the binary components can be customized for a particular deployment environment.

3. *Deployment* The component model includes a specification of how components should be packaged for deployment as independent, executable entities. Because components are independent entities, they have to be packaged with all supporting software that is not provided by the component infrastructure, or is not defined in a 'requires' interface. Deployment information includes information about the contents of a package and its binary organization.

   Inevitably, as new requirements emerge, components will have to be changed or replaced. The component model may therefore include rules governing when and how component replacement is allowed. Finally, the component model may

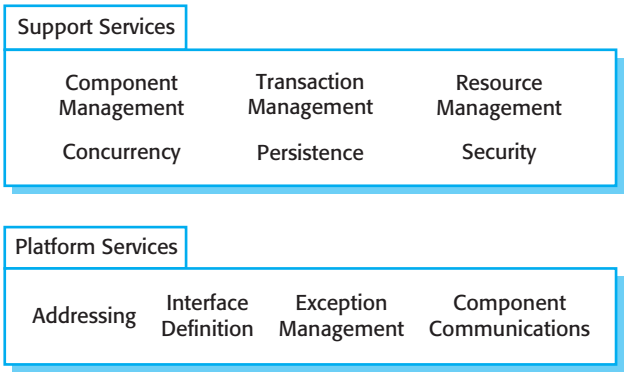


Figure 17.5 Middleware services defined in a component model

define the component documentation that should be produced. This is used to find the component and to decide whether it is appropriate.

For components that are implemented as program units rather than external services, the component model sets out the services to be provided by the middleware that supports the executing components. Weinreich and Sametinger (2001) use the analogy of an operating system to explain component models. An operating system provides a set of generic services that can be used by applications. A component model implementation provides comparable shared services for components. Figure 17.5 shows some of the services that may be provided by an implementation of a component model.

The services provided by a component model implementation fall into two categories:

1. Platform services, which enable components to communicate and interoperate in a distributed environment. These are the fundamental services that must be available in all component-based systems.
2. Support services, which are common services that are likely to be required by many different components. For example, many components require authentication to ensure that the user of component services is authorized. It makes sense to provide a standard set of middleware services for use by all components. This reduces the costs of component development and potential component incompatibilities can be avoided.

The middleware implements the component services and provides interfaces to these services. To make use of the services provided by a component model infrastructure, you can think of the components as being deployed in a 'container'. A container is an implementation of the support services plus a definition of the interfaces that a component must provide to integrate it with the container. Including the component in the container means that the component can access the support services and the container can access the component interfaces. When in use, the component interfaces themselves are not accessed directly by other components;

rather, they are accessed through a container interface that invokes code to access the interface of the embedded component.

Containers are large and complex and, when you deploy a component in a container, you get access to all middleware services. However, simple components may not need all of the facilities offered by the supporting middleware. The approach taken in web services to common service provision is therefore rather different. For web services, standards have been defined for common services such as transaction management and security and these standards have been implemented as program libraries. If you are implementing a service component, you only use the common services that you need.

17.2 CBSE processes

CBSE processes are software processes that support component-based software engineering. They take into account the possibilities of reuse and the different process activities involved in developing and using reusable components. Figure 17.6 (Kotonya, 2003) presents an overview of the processes in CBSE. At the highest level, there are two types of CBSE processes:

1. *Development for reuse* This process is concerned with developing components or services that will be reused in other applications. It usually involves generalizing existing components.
2. *Development with reuse* This is the process of developing new applications using existing components and services.

These processes have different objectives and therefore, include different activities. In the development for reuse process, the objective is to produce one or more reusable components. You know the components that you will be working with and you have access to their source code to generalize them. In development with reuse, you don't know what components are available, so you need to discover these components and design your system to make the most effective use of them. You may not have access to the component source code.

You can see from Figure 17.6 that the basic processes of CBSE with and for reuse have supporting processes that are concerned with component acquisition, component management, and component certification:

1. Component acquisition is the process of acquiring components for reuse or development into a reusable component. It may involve accessing locally developed components or services or finding these components from an external source.
2. Component management is concerned with managing a company's reusable components, ensuring that they are properly cataloged, stored, and made available for reuse.

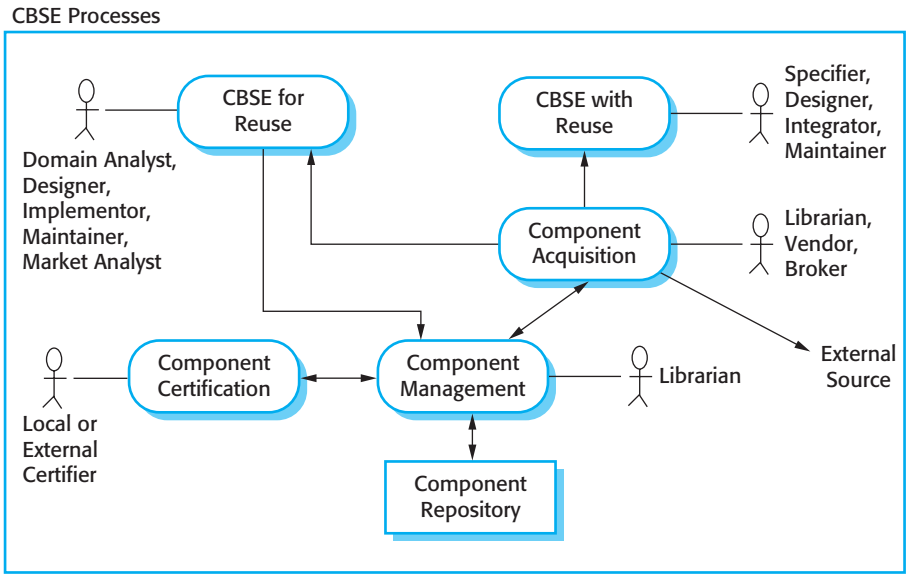


Figure 17.6 CBSE processes

3. Component certification is the process of checking a component and certifying that it meets its specification.

Components maintained by an organization may be stored in a component repository that includes both the components and information about their use.

17.2.1 CBSE for reuse

CBSE for reuse is the process of developing reusable components and making them available for reuse through a component management system. The vision of early supporters of CBSE (Szyperski, 2002) was that a thriving component marketplace would develop. There would be specialist component providers and component vendors who would organize the sale of components from different developers. Software developers would buy components to include in a system or pay for services as they were used. However, this vision has not been realized. There are relatively few component suppliers and buying components is uncommon. At the time of writing, the service market is also undeveloped although there are predictions that it will expand significantly over the next few years.

Consequently, CBSE for reuse is most likely to take place within an organization that has made a commitment to reuse-driven software engineering. They wish to exploit the software assets that have been developed in different parts of the company. However, these internally developed components are not usually reusable without change. They often include application-specific features and interfaces that are unlikely to be required in other programs where the component is reused.

To make components reusable, you have to adapt and extend the application-specific components to create more generic and therefore more reusable versions. Obviously, this adaptation has an associated cost. Thus you have to decide first, whether a component is likely to be reused and second, whether the cost savings from future reuse justify the costs of making the component reusable.

To answer the first of these questions, you have to decide whether or not the component implements one or more stable domain abstractions. Stable domain abstractions are fundamental elements of the application domain that change slowly. For example, in a banking system, domain abstractions might include accounts, account holders, and statements. In a hospital management system, domain abstractions might include patients, treatments, and nurses. These domain abstractions are sometimes called 'business objects'. If the component is an implementation of a commonly used domain abstraction or group of related business objects, it can probably be reused.

To answer the question about the cost effectiveness, you have to assess the costs of changes that are required to make the component reusable. These costs are the costs of component documentation, component validation, and making the component more generic. Changes that you may make to a component to make it more reusable include:

- removing application-specific methods;
- changing names to make them more general;
- adding methods to provide more complete functional coverage;
- making exception handling consistent for all methods;
- adding a 'configuration' interface to allow the component to be adapted to different situations of use;
- integrating required components to increase independence.

The problem of exception handling is a particularly difficult one. Components should not handle exceptions themselves, because each application will have its own requirements for exception handling. Rather, the component should define what exceptions can arise and should publish these as part of the interface. For example, a simple component implementing a stack data structure should detect and publish stack overflow and stack underflow exceptions. In practice, however, there are two problems with this:

1. Publishing all exceptions leads to bloated interfaces that are harder to understand. This may put off potential users of the component.
2. The operation of the component may depend on local exception handling, and changing this may have serious implications for the functionality of the component.

Mili et al. (2002) discuss ways of estimating the costs of making a component reusable and the returns from that investment. The benefits of reusing rather than redeveloping a component are not simply productivity gains. There are also quality gains, because a reused component should be more dependable, and time-to-market gains. These are the increased returns that accrue from deploying the software more quickly. Mili et al. present various formulas for estimating these gains, as does the COCOMO model discussed in Chapter 23 (Boehm, et al., 2000). However, the parameters of these formulas are difficult to estimate accurately, and the formulas must be adapted to local circumstances, making them difficult to use. I suspect that few software project managers use these models to estimate the return on investment from component reusability.

Obviously, whether or not a component is reusable depends on its application domain and functionality. As you add generality to a component, you increase its reusability. However, this normally means that the component has more operations and is more complex, which makes the component harder to understand and use.

There is, therefore, an inevitable trade-off between reusability and usability of a component. To make a component reusable you have to provide a set of generic interfaces with operations that cater to all of the ways in which the component could be used. Making the component usable means providing a simple, minimal interface that is easy to understand. Reusability adds complexity and hence reduces component understandability. It is therefore more difficult to decide when and how to reuse that component. When designing a reusable component, you must, therefore, find a compromise between generality and understandability.

A potential source of components is existing legacy systems. As I discussed in Chapter 9, these are systems that fulfill an important business function but are written using obsolete software technologies. Because of this, it may be difficult to use them with new systems. However, if you convert these old systems to components, their functionality can be reused in new applications.

Of course, these legacy systems do not normally have clearly defined ‘requires’ and ‘provides’ interfaces. To make these components reusable, you have to create a wrapper that defines the component interfaces. The wrapper hides the complexity of the underlying code and provides an interface for external components to access services that are provided. Although this wrapper is a fairly complex piece of software, the cost of wrapper development is often much less than the cost of reimplementing the legacy system. I discuss this approach in more detail in Chapter 19, where I explain how the features in a legacy system can be accessed through services.

Once you have developed and tested a reusable component or service, this then has to be managed for future reuse. Management involves deciding how to classify the component so that it can be discovered, making the component available either in a repository or as a service, maintaining information about the use of the component and keeping track of different component versions. If the component is open source, you may make it available in a public repository such as Sourceforge. If it is intended for use in a company, then you may use an internal repository system.

A company with a reuse program may carry out some form of component certification before the component is made available for reuse. Certification means that

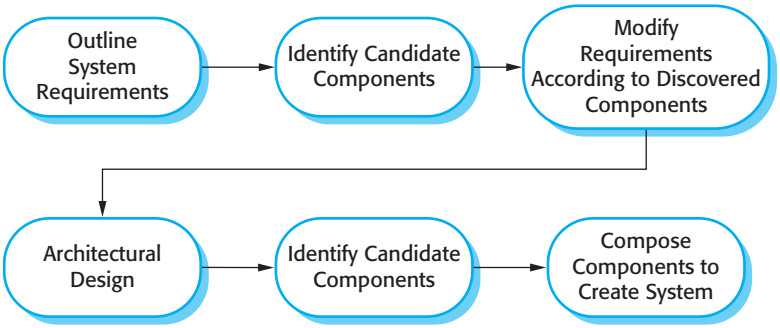


Figure 17.7 CBSE with reuse

someone apart from the developer checks the quality of the component. They test the component and certify that it has reached an acceptable quality standard, before it is made available for reuse. However, this can be an expensive process and many companies simply leave testing and quality checking to the component developers.

17.2.2 CBSE with reuse

The successful reuse of components requires a development process tailored to CBSE. The CBSE with reuse process has to include activities that find and integrate reusable components. The structure of such a process was discussed in Chapter 2 and Figure 17.7 shows the principal activities within that process. Some of the activities within this process, such as the initial discovery of user requirements, are carried out in the same way as in other software processes. However, the essential differences between CBSE with reuse and software processes for original software development are:

1. The user requirements are initially developed in outline rather than in detail, and stakeholders are encouraged to be as flexible as possible in defining their requirements. Requirements that are too specific limit the number of components that could meet these requirements. However, unlike incremental development, you need a complete set of requirements so that you can identify as many components as possible for reuse.
2. Requirements are refined and modified early in the process depending on the components available. If the user requirements cannot be satisfied from available components, you should discuss the related requirements that can be supported. Users may be willing to change their minds if this means cheaper or quicker system delivery.
3. There is a further component search and design refinement activity after the system architecture has been designed. Some apparently usable components may turn out to be unsuitable or do not work properly with other chosen components. Although not shown in Figure 17.7, this implies that further requirements changes may be necessary.

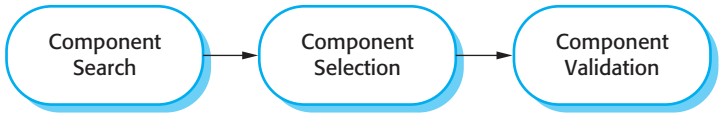


Figure 17.8 The component identification process

4. Development is a composition process where the discovered components are integrated. This involves integrating the components with the component model infrastructure and, often, developing adaptors that reconcile the interfaces of incompatible components. Of course, additional functionality may also be required over and above that provided by reused components.

The architectural design stage is particularly important. Jacobson et al. (1997) found that defining a robust architecture is critical for successful reuse. During the architectural design activity, you may choose a component model and implementation platform. However, many companies have a standard development platform (e.g., .NET) so the component model is pre-determined. As I discussed in Chapter 6, you also establish the high-level organization of the system at this stage and make decisions about system distribution and control.

An activity that is unique to the CBSE process is identifying candidate components or services for reuse. This involves a number of subactivities, as shown in Figure 17.8. Initially, your focus should be on search and selection. You need to convince yourself that there are components available to meet your requirements. Obviously, you should do some initial checking that the component is suitable but detailed testing may not be required. In the later stage, after the system architecture has been designed, you should spend more time on component validation. You need to be confident that the identified components are really suited to your application; if not, then you have to repeat the search and selection processes.

The first step in identifying components is to look for components that are available locally or from trusted suppliers. As I said in the previous section, there are relatively few component vendors so you are therefore most likely to be looking for components that have been developed in your own company. Software development companies can build their own database of reusable components without the risks inherent in using components from external suppliers. Alternatively, you may decide to search code libraries available on the Web, such as Sourceforge or Google Code, to see if source code for the component that you need is available. If you are looking for services, then there are a number of specialized web search engines available that can discover public web services.

Once the component search process has identified possible components, you have to select candidate components for assessment. In some cases, this will be a straightforward task. Components on the list will directly implement the user requirements and there will not be competing components that match these requirements. In other cases, however, the selection process is much more complex. There will not be a clear mapping of requirements onto components and you may find that several components have to be integrated to meet a specific requirement or group of requirements.

The Ariane 5 launcher Failure

While developing the Ariane 5 space launcher, the designers decided to reuse the inertial reference software that had performed successfully in the Ariane 4 launcher. The inertial reference software maintains the stability of the rocket. They decided to reuse this without change (as you would do with components), although it included additional functionality that was not required in Ariane 5.

In the first launch of Ariane 5, the inertial navigation software failed and the rocket could not be controlled. Ground controllers instructed the launcher to self-destruct and the rocket and its payload were destroyed. The cause of the problem was an unhandled exception when a conversion of a fixed-point number to an integer resulted in a numeric overflow. This caused the run-time system to shut down the inertial reference system and launcher stability could not be maintained. The fault had never occurred in Ariane 4 because it had less powerful engines and the value that was converted could not be large enough for the conversion to overflow.

The fault occurred in code that was not required for Ariane 5. The validation tests for the reused software were based on Ariane 5 requirements. Because there were no requirements for the function that failed, no tests were developed. Consequently, the problem with the software was never discovered during launch simulation tests.

Figure 17.9 An example of validation failure with reused software

You, therefore, have to decide which component compositions provide the best coverage of the requirements.

Once you have selected components for possible inclusion in a system, you should then validate them to check that they behave as advertised. The extent of the validation required depends on the source of the components. If you are using a component that has been developed by a known and trusted source, you may decide that component testing is unnecessary. You simply test the component when it is integrated with other components. On the other hand, if you are using a component from an unknown source, you should always check and test that component before including it in your system.

Component validation involves developing a set of test cases for a component (or, possibly, extending test cases supplied with that component) and developing a test harness to run component tests. The major problem with component validation is that the component specification may not be sufficiently detailed to allow you to develop a complete set of component tests. Components are usually specified informally, with the only formal documentation being their interface specification. This may not include enough information for you to develop a complete set of tests that would convince you that the component's advertised interface is what you require.

As well as testing that a component for reuse does what you require, you may also have to check that the component does not include any malicious code or functionality that you don't need. Professional developers rarely use components from untrusted sources, especially if these sources do not provide source code. Therefore, the malicious code problem does not usually arise. However, components may often contain functionality that you don't need and you have to check that this functionality will not interfere with your use of the component.

The problem with unnecessary functionality is that it may be activated by the component itself. This can slow down the component, cause it to produce surprising results or, in some cases, cause serious system failures. Figure 17.9 summarizes a situation where unnecessary functionality in a reused system caused a catastrophic software failure.

The problem in the Ariane 5 launcher arose because the assumptions made about the software for Ariane 4 were invalid for Ariane 5. This is a general problem with reusable components. They are originally implemented for an application environment and, naturally, embed assumptions about that environment. These assumptions are rarely documented so, when the component is reused, it is impossible to derive tests to check if the assumptions are still valid. If you are reusing a component in a different environment, you may not discover the embedded environmental assumptions until you use the component in an operational system.

17.3 Component composition

Component composition is the process of integrating components with each other, and with specially written 'glue code' to create a system or another component. There are several different ways in which you can compose components, as shown in Figure 17.10. From left to right these diagrams illustrate sequential composition, hierarchical composition and additive composition. In the discussion below, I assume that you are composing two components (A and B) to create a new component:

1. Sequential composition is situation (a) in Figure 17.10. You create a new component from 2 existing components by calling the existing components in sequence. You can think of the composition as a composition of the 'provides interfaces'. That is, the services offered by component A are called and the results returned by A are then used in the call to the services offered by component B. The components do not call each other in sequential composition. Some extra glue code is required to call the component services in the right order and to ensure that the results delivered by component A are compatible with the inputs expected by component B. The 'provides' interface of the composition depends on the combined functionality of A and B but will not normally be a composition of their 'provides interfaces'. This type of composition may be used with components that are program elements or components that are services.
2. Hierarchical composition is situation (b) in Figure 17.10. This type of composition occurs when one component calls directly on the services provided by another component. The called component provides the services that are required by the calling component. Therefore, the 'provides' interface of the called component must be compatible with the 'requires' interface of the calling component. Component A calls on component B directly and, if their interfaces match, there may be no need for additional code. However, if there is a mismatch between the 'requires' interface of A and the 'provides' interface of B, then some conversion code may be required. As services do not have a 'requires' interface, this mode of composition is not used when components are implemented as web services.
3. Additive composition corresponds to situation (c) in Figure 17.10. This occurs when two or more components are put together (added) to create a new component,

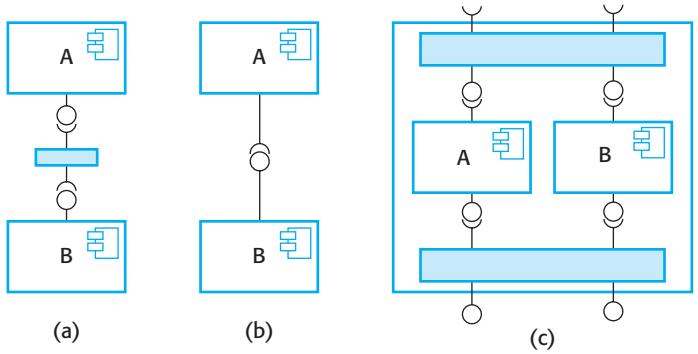


Figure 17.10 Types of component composition

which combines their functionality. The 'provides' interface and 'requires' interface of the new component is a combination of the corresponding interfaces in components A and B. The components are called separately through the external interface of the composed component. A and B are not dependent and do not call each other. This type of composition may be used with components that are program units or components that are services.

You might use all the forms of component composition when creating a system. In all cases, you may have to write 'glue code' that links the components. For example, for sequential composition, the output of component A typically becomes the input to component B. You need intermediate statements that call component A, collect the result, and then call component B with that result as a parameter. When one component calls another, you may need to introduce an intermediate component that ensures that the 'provides' interface and the 'requires' interface are compatible.

When you write new components especially for composition, you should design the interfaces of these components so that they are compatible with other components in the system. You can therefore easily compose these components into a single unit. However, when components are developed independently for reuse, you will often be faced with interface incompatibilities. This means that the interfaces of the components that you wish to compose are not the same. Three types of incompatibility can occur:

1. *Parameter incompatibility* The operations on each side of the interface have the same name but their parameter types or the number of parameters are different.
2. *Operation incompatibility* The names of the operations in the 'provides' and 'requires' interfaces are different.
3. *Operation incompleteness* The 'provides' interface of a component is a subset of the 'requires' interface of another component or vice versa.

In all cases, you tackle the problem of incompatibility by writing an adaptor that reconciles the interfaces of the two components being reused. An adaptor component converts one interface to another. The precise form of the adaptor depends on the type

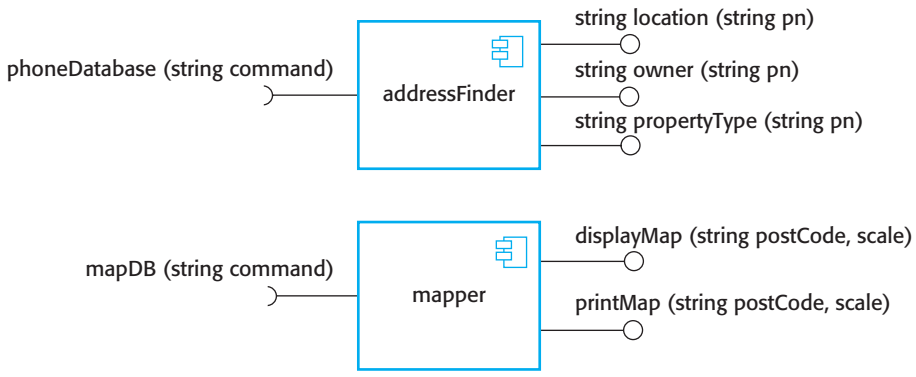


Figure 17.11 Components with incompatible interfaces

of composition. Sometimes, as in the next example, the adaptor takes a result from one component and converts it into a form where it can be used as an input to another. In other cases, the adaptor may be called by component A as a proxy for component B. This situation occurs if A wishes to call B but the details of the 'requires' interface of A do not match the details of the 'provides' interface of B. The adaptor reconciles these differences by converting its input parameters from A into the required input parameters for B. It then calls B to deliver the services required by A.

To illustrate adaptors, consider the two components shown in Figure 17.11, whose interfaces are incompatible. These might be part of a system used by the emergency services. When the emergency operator takes a call, the phone number is input to the addressFinder component to locate the address. Then, using the mapper component, the operator prints a map to be sent to the vehicle dispatched to the emergency. In fact, the components would have more complex interfaces than those shown here, but the simplified version illustrates the concept of an adaptor.

The first component, addressFinder, finds the address that matches a phone number. It can also return the owner of the property associated with the phone number and the type of property. The mapper component takes a post code (in the United States, a standard ZIP code with the additional four digits identifying property location) and displays or prints a street map of the area around that code at a specified scale.

These components are composable in principle because the property location includes the post or ZIP code. However, you have to write an adaptor component called postCodeStripper that takes the location data from addressFinder and strips out the post code. This post code is then used as an input to mapper and the street map is displayed at a scale of 1:10,000. The following code, which is an example of sequential composition, illustrates the sequence of calls that is required to implement this:

```
address = addressFinder.location (phonenumber) ;
postCode = postCodeStripper.getPostCode (address) ;
mapper.displayMap(postCode, 10000) ;
```

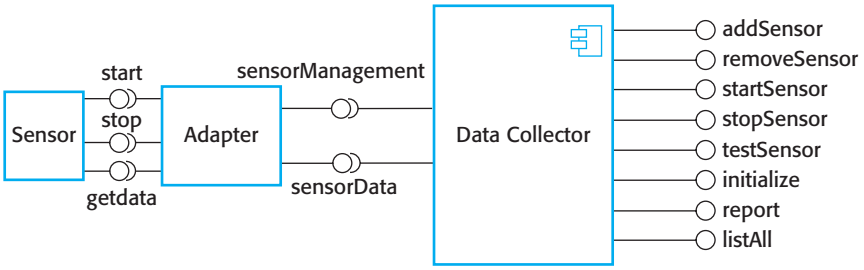


Figure 17.12 An adaptor linking a data collector and a sensor

Another case in which an adaptor component may be used is in hierarchical composition, where one component wishes to make use of another but there is an incompatibility between the 'provides' interface and 'requires' interface of the components in the composition. I have illustrated the use of an adaptor in Figure 17.12 where an adaptor is used to link a data collector and a sensor component. These could be used in the implementation of a wilderness weather station system, as discussed in Chapter 7.

The sensor and data collector components are composed using an adapter that reconciles the 'requires' interface of the data collection component with the 'provides' interface of the sensor component. The data collector component has been designed with a generic 'requires' interface that supports sensor data collection and sensor management. For each of these operations, the parameter is a text string representing the specific sensor commands. For example, to issue a collect command, you would say sensorData("collect"). As I have shown in Figure 17.12, the sensor itself has separate operations such as start, stop, and getdata.

The adaptor parses the input string, identifies the command (e.g., collect) and then calls Sensor.getdata to collect the sensor value. It then returns the result (as a character string) to the data collector component. This interface style means that the data collector can interact with different types of sensor. A separate adaptor, which converts the sensor commands from Data collector to the actual sensor interface, is implemented for each type of sensor.

The above discussion of component composition assumes you can tell from the component documentation whether or not interfaces are compatible. Of course, the interface definition includes the operation name and parameter types, so you can make some assessment of the compatibility from this. However, you depend on the component documentation to decide whether the interfaces are semantically compatible.

To illustrate this problem, consider the composition shown in Figure 17.13. These components are used to implement a system that downloads images from a digital camera and stores them in a photograph library. The system user can provide additional information to describe and catalog the photograph. To avoid clutter,

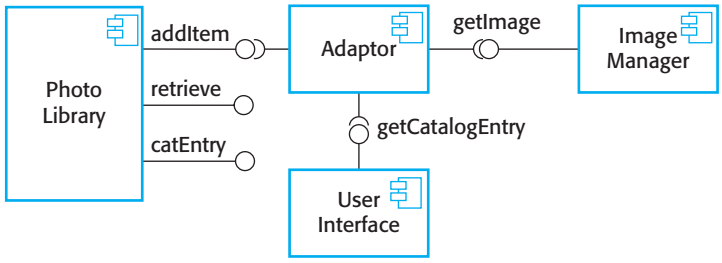


Figure 17.13 Photo library composition

I have not shown all interface methods here. Rather, I simply show the methods that are needed to illustrate the component documentation problem. The methods in the interface of Photo Library are:

```
public void addItem (Identifier pid ; Photograph p; CatalogEntry
photodesc) ;
public Photograph retrieve (Identifier pid) ;
public CatalogEntry catEntry (Identifier pid);
```

Assume that the documentation for the addItem method in Photo Library is:

This method adds a photograph to the library and associates the photograph identifier and catalogue descriptor with the photograph.

This description appears to explain what the component does, but consider the following questions:

- What happens if the photograph identifier is already associated with a photograph in the library?
- Is the photograph descriptor associated with the catalog entry as well as the photograph? That is, if you delete the photograph, do you also delete the catalog information?

There is not enough information in the informal description of addItem to answer these questions. Of course, it is possible to add more information to the natural language description of the method, but in general, the best way to resolve ambiguities is to use a formal language to describe the interface. The specification shown in Figure 17.14 is part of the description of the interface of Photo Library that adds information to the informal description.

The specification in Figure 17.14 uses pre- and post-conditions that are defined in a notation based on the object constraint language (OCL), which is part of the UML (Warmer and Kleppe, 2003). OCL is designed to describe constraints in UML object models; it allows you to express predicates that must always be true, that must be

```
— The context keyword names the component to which the conditions apply

context addItem

— The preconditions specify what must be true before execution of addItem
pre:     PhotoLibrary.libSize() > 0
         PhotoLibrary.retrieve(pid) = null

— The postconditions specify what is true after execution
post:    libSize () = libSize()@pre + 1
         PhotoLibrary.retrieve(pid) = p
         PhotoLibrary.catEntry(pid) = photodesc

context delete

pre:     PhotoLibrary.retrieve(pid) <>null ;

post:    PhotoLibrary.retrieve(pid) = null
         PhotoLibrary.catEntry(pid) = PhotoLibrary.catEntry(pid)@pre
         PhotoLibrary.libSize() = libSize()@pre—1
```

Figure 17.14 The OCL description of the Photo Library interface

true before a method has executed; and that must be true after a method has executed. These are invariants, pre-conditions, and post-conditions. To access the value of a variable before an operation, you add @pre after its name. Therefore, using age as an example:

```
age = age@pre + 1
```

This statement means that the value of age after an operation is one more than it was before that operation.

OCL-based approaches are increasingly used to add semantic information to UML models, and OCL descriptions may be used to drive code generators in model-driven engineering. The general approach has been derived from Meyer's Design by Contract approach (Meyer, 1992), in which the interfaces and obligations of communicating objects are formally specified and enforced by the run-time system. Meyer suggests that using Design by Contract is essential if we are to develop trusted components (Meyer, 2003).

Figure 17.14 includes a specification for the addItem and delete methods in Photo Library. The method being specified is indicated by the keyword context and the pre- and post-conditions by the keywords pre and post. The pre-conditions for addItem state that:

1. There must not be a photograph in the library with the same identifier as the photograph to be entered.
2. The library must exist—assume that creating a library adds a single item to it so that the size of a library is always greater than zero.

3. The post-conditions for addItem state that:

   The size of the library has increased by 1 (so only a single entry has been made).

   If you retrieve using the same identifier, then you get back the photograph that you added.

   If you look up the catalogue using that identifier, you get back the catalogue entry that you made.

The specification of delete provides further information. The pre-condition states that to delete an item, it must be in the library and, after deletion, the photo can no longer be retrieved and the size of the library is reduced by 1. However, delete does not delete the catalogue entry—you can still retrieve it after the photo has been deleted. The reason for this is that you may wish to maintain information in the catalog about why a photo was deleted, its new location, and so on.

When you create a system by composing components, you may find that there are potential conflicts between functional and non-functional requirements, the need to deliver a system as quickly as possible and the need to create a system that can evolve as requirements change. The decisions where you may have to take trade-offs into account are:

1. What composition of components is most effective for delivering the functional requirements for the system?
2. What composition of the components will make it easier to adapt the composite component when its requirements change?
3. What will be the emergent properties of the composed system? These are properties such as performance and dependability. You can only assess these once the complete system is implemented.

Unfortunately, there are many situations where the solutions to the composition problems may conflict. For example, consider a situation such as that illustrated in Figure 17.15, where a system can be created through two alternative compositions. The system is a data collection and reporting system where data is collected from different sources, stored in a database and then different reports summarizing that data are produced.

Here, there is a potential conflict between adaptability and performance. Composition (a) is more adaptable but composition (b) is perhaps faster and more reliable. The advantages of composition (a) are that reporting and data management are separate, so there is more flexibility for future change. The data management system could be replaced and, if reports are required that the current reporting component cannot produce, that component can also be replaced without having to change the data management component.

In composition (b), a database component with built-in reporting facilities (e.g., Microsoft Access) is used. The key advantage of composition (b) is that there are fewer components, so this will be a faster implementation because there are no component

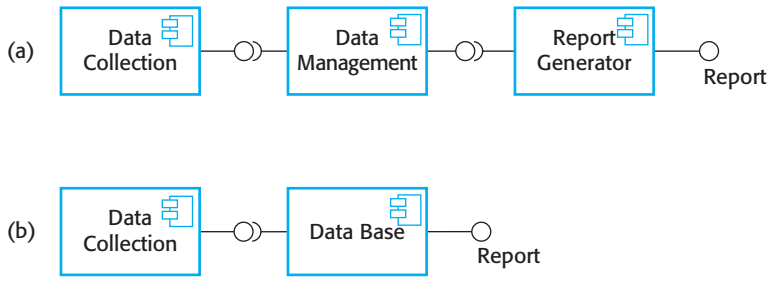


Figure 17.15 Data collection and report generation components

communication overheads. Furthermore, data integrity rules that apply to the database will also apply to reports. These reports will not be able to combine data in incorrect ways. In composition (a), there are no such constraints so errors in reports could occur.

In general, a good composition principle to follow is the principle of separation of concerns. That is, you should try to design your system in such a way that each component has a clearly defined role and that, ideally, these roles should not overlap. However, it may be cheaper to buy one multi-functional component rather than two or three separate components. Furthermore, there may be dependability or performance penalties when multiple components are used.

KEY POINTS

- Component-based software engineering is a reuse-based approach to defining, implementing, and composing loosely coupled independent components into systems.
- A component is a software unit whose functionality and dependencies are completely defined by a set of public interfaces. Components can be composed with other components without knowledge of their implementation and can be deployed as an executable unit.
- Components may be implemented as program units that are included in a system or as external services that are referenced from within a system.
- A component model defines a set of standards for components, including interface standards, usage standards, and deployment standards. The implementation of the component model provides a set of common services that may be used by all components.
- During the CBSE process, you have to interleave the processes of requirements engineering and system design. You have to trade off desirable requirements against the services that are available from existing reusable components.
- Component composition is the process of 'wiring' components together to create a system. Types of composition include sequential composition, hierarchical composition, and additive composition.

- When composing reusable components that have not been written for your application, you may need to write adaptors or 'glue code' to reconcile the different component interfaces.
- When choosing compositions, you have to consider the required functionality of the system, the non-functional requirements and the ease with which one component can be replaced when the system is changed.

FURTHER READING

Component-based Software Engineering: Putting the Pieces Together. This book is a collection of papers from various authors on different aspects of CBSE. Like all collections, it is rather mixed but it has better coverage of general issues of software engineering with components than Szyperski's book. (G. T. Heineman and W. T. Councill, Addison-Wesley, 2001.)

Component Software: Beyond Object-Oriented Programming, 2nd ed. This updated edition of the first book on CBSE covers technical and nontechnical issues in CBSE. It has more detail on specific technologies than Heineman and Councill's book and includes a thorough discussion of market issues. (C. Szyperski, Addison-Wesley, 2002.)

'Specification, Implementation and Deployment of Components'. A good introduction to the fundamentals of CBSE. The same issue of the *CACM* includes articles on components and component-based development. (I. Crnkovic, B. Hnich, T. Jonsson and Z. Kiziltan, *Comm. ACM,* **45** (10), October 2002.) http://dx.doi.org/10.1145/570907.570928.

'Software Component Models'. This is a comprehensive discussion of commercial and research component models that classifies these models and explains the differences between them. (K-K. Lau and Z. Wang, *IEEE Transactions on Software Engineering,* **33** (10), October 2007.) http://dx.doi.org/10.1109/TSE.2007.70726.

EXERCISES

17.1. Why is it important that all component interactions are defined through 'requires' and 'provides' interfaces?

17.2. The principle of component independence means that it ought to be possible to replace one component with another that is implemented in a completely different way. Using an example, explain how such component replacement could have undesired consequences and may lead to system failure.

17.3. What are the fundamental differences between components as program elements and components as services?

17.4. Why is it important that components should be based on a standard component model?

17.5. Using an example of a component that implements an abstract data type such as a stack or a list, show why it is usually necessary to extend and adapt components for reuse.

17.6. Explain why it is difficult to validate a reusable component without the component source code. In what ways would a formal component specification simplify the problems of validation?

17.7. Design the 'provides' interface and the 'requires' interface of a reusable component that may be used to represent a patient in the MHC-PMS.

17.8. Using examples, illustrate the different types of adaptor needed to support sequential composition, hierarchical composition, and additive composition.

17.9. Design the interfaces of components that might be used in a system for an emergency control room. You should design interfaces for a call-logging component that records calls made, and a vehicle discovery component that, given a post code (zip code) and an incident type, finds the nearest suitable vehicle to be despatched to the incident.

17.10. It has been suggested that an independent certification authority should be established. Vendors would submit their components to this authority, which would validate that the component was trustworthy. What would be the advantages and disadvantages of such a certification authority?

REFERENCES

Boehm, B. W., Abts, C., Brown, A. W., Chulani, S., Clark, B. K., Horowitz, E., Madachy, R., Reifer, D. and Steece, B. (2000). *Software Cost Estimation with COCOMO II*. Upper Saddle River, NJ.: Prentice Hall.

Councill, W. T. and Heineman, G. T. (2001). 'Definition of a Software Component and its Elements'. In *Component-based Software Engineering*. Heineman, G. T. and Councill, W. T. (ed.). Boston: Addison-Wesley, 5–20.

Jacobson, I., Griss, M. and Jonsson, P. (1997). *Software Reuse*. Reading, Mass.: Addison-Wesley.

Kotonya, G. (2003). 'The CBSE Process: Issues and Future Visions'. *Proc. 2nd CBSEnet workshop,* Budapest, Hungary.

Lau, K.-K. and Wang, Z. (2007). 'Software Component Models'. *IEEE Trans. on Software Eng.*, **33** (10), 709–24.

Meyer, B. (1992). 'Design by Contract'. *IEEE Computer,* **25** (10), 40–51.

Meyer, B. (2003). 'The Grand Challenge of Trusted Components'. *ICSE 25: Int. Conf. on Software Eng.*, Portland, Oregon: IEEE Press.

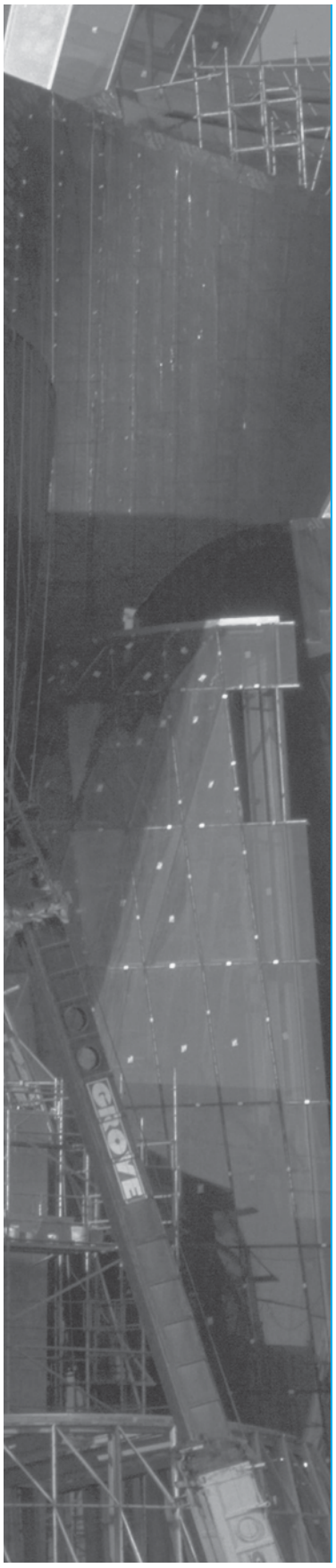
Mili, H., Mili, A., Yacoub, S. and Addy, E. (2002). *Reuse-based Software Engineering*. New York: John Wiley & Sons.

Pope, A. (1997). *The CORBA Reference Guide: Understanding the Common Object Request Broker Architecture*. Harlow, UK: Addison-Wesley.

Szyperski, C. (2002). *Component Software: Beyond Object-oriented Programming, 2nd ed*. Harlow, UK: Addison-Wesley.

Warmer, J. and Kleppe, A. (2003). *The Object Constraint Language: Getting your models ready for MDA*. Boston: Addison-Wesley.

Weinreich, R. and Sametinger, J. (2001). 'Component Models and Component Services: Concepts and Principles'. In *Component-Based Software Engineering*. Heineman, G. T. and Councill, W. T. (ed.). Boston: Addison-Wesley, 33–48.



18

Distributed software engineering

Objectives

The objective of this chapter is to introduce distributed systems engineering and distributed systems architectures. When you have read this chapter, you will:

- know the key issues that have to be considered when designing and implementing distributed software systems;
- understand the client–server computing model and the layered architecture of client–server systems;
- have been introduced to commonly used patterns for distributed systems architectures and know the types of system for which each architecture is most applicable;
- understand the notion of software as a service, providing web-based access to remotely deployed application systems.

Contents

18.1 Distributed systems issues
18.2 Client–server computing
18.3 Architectural patterns for distributed systems
18.4 Software as a service

Virtually all large computer-based systems are now distributed systems. A distributed system is one involving several computers, in contrast with centralized systems where all of the system components execute on a single computer. Tanenbaum and Van Steen (2007) define a distributed system to be:

> *". . .a collection of independent computers that appears to the user as a single coherent system."*

Obviously, the engineering of distributed systems has a great deal in common with the engineering of any other software. However, there are specific issues that have to be taken into account when designing this type of system. These arise because the system components may be running on independently managed computers and they communicate across a network.

Coulouris et al. (2005) identify the following advantages of using a distributed approach to systems development:

1. *Resource sharing* A distributed system allows the sharing of hardware and software resources—such as disks, printers, files, and compilers—that are associated with computers on a network.
2. *Openness* Distributed systems are normally open systems, which means that they are designed around standard protocols that allow equipment and software from different vendors to be combined.
3. *Concurrency* In a distributed system, several processes may operate at the same time on separate computers on the network. These processes may (but need not) communicate with each other during their normal operation.
4. *Scalability* In principle at least, distributed systems are scalable in that the capabilities of the system can be increased by adding new resources to cope with new demands on the system. In practice, the network linking the individual computers in the system may limit the system scalability.
5. *Fault tolerance* The availability of several computers and the potential for replicating information means that distributed systems can be tolerant of some hardware and software failures (see Chapter 13). In most distributed systems, a degraded service can be provided when failures occur; complete loss of service only occurs when there is a network failure.

For large-scale organizational systems, these advantages mean that distributed systems have largely replaced mainframe legacy systems that were developed in the 1990s. However, there are many personal computer application systems (e.g., photo editing systems) that are not distributed and which run on a single computer system. Most embedded systems are also single processor systems.

Distributed systems are inherently more complex than centralized systems. This makes them more difficult to design, implement, and test. It is harder to understand the emergent properties of distributed systems because of the complexity of the

interactions between system components and the system infrastructure. For example, rather than the performance of the system being dependent on the execution speed of one processor, it depends on the network bandwidth, the network load, and the speed of all of the computers that are part of the system. Moving resources from one part of the system to another can significantly affect the system's performance.

Furthermore, as all users of the WWW know, distributed systems are unpredictable in their response. The response time depends on the overall load on the system, its architecture and the network load. As all of these may change over a short time, the time taken to respond to a user request may vary dramatically from one request to another.

The most important development that has affected distributed software systems in the past few years is the service-oriented approach. Much of this chapter focuses on general issues of distributed systems, but I cover the notion of applications deployed as services in Section 18.4. This complements the material in Chapter 19, which focuses on services as components in a service-oriented architecture, and more general issues of service-oriented software engineering.

18.1 Distributed systems issues

As I discussed in the introduction to this chapter, distributed systems are more complex than systems that run on a single processor. This complexity arises because it is practically impossible to have a top-down model of control of these systems. The nodes in the system that deliver functionality are often independent systems with no single authority in charge of them. The network connecting these nodes is a separately managed system. It is a complex system in its own right and cannot be controlled by the owners of systems using the network. There is therefore an inherent unpredictability in the operation of distributed systems that has to be taken into account by the system designer.

Some of the most important design issues that have to be considered in distributed systems engineering are:

1. *Transparency* To what extent should the distributed system appear to the user as a single system? When it is useful for users to understand that the system is distributed?
2. *Openness* Should a system be designed using standard protocols that support interoperability or should more specialized protocols be used that restrict the freedom of the designer?
3. *Scalability* How can the system be constructed so that it is scaleable? That is, how can the overall system be designed so that its capacity can be increased in response to increasing demands made on the system?
4. *Security* How can usable security policies be defined and implemented that apply across a set of independently managed systems?



CORBA – Common Object Request Broker Architecture

CORBA is a well-known specification for a middleware system that was developed in the 1990s by the Object Management Group. It was intended as an open standard that would allow the development of middleware to support distributed component communications and execution, plus provide a set of standard services that could be used by these components.

Several implementations of CORBA were produced but the system never achieved critical mass. Users preferred proprietary systems or moved to service-oriented architectures.

http://www.SoftwareEngineering-9.com/Web/DistribSys/Corba.html

5. *Quality of service* How should the quality of service that is delivered to system users be specified and how should the system be implemented to deliver an acceptable quality of service to all users?
6. *Failure management* How can system failures be detected, contained (so that they have minimal effects on other components in the system), and repaired?

In an ideal world, the fact that a system is distributed would be transparent to users. This means that users would see the system as a single system whose behavior is not affected by the way that the system is distributed. In practice, this is impossible to achieve. Central control of a distributed system is impossible and, as a result, individual computers in a system may behave differently at different times. Furthermore, because it always takes a finite length of time for signals to travel across a network, network delays are unavoidable. The length of these delays depends on the location of resources in the system, the quality of the user's network connection, and the network load.

The design approach to achieving transparency depends on creating abstractions of the resources in a distributed system so that the physical realization of these resources can be changed without having to make changes in the application system. Middleware (discussed in Section 18.1.2) is used to map the logical resources referenced by a program onto the actual physical resources, and to manage the interactions between these resources.

In practice, it is impossible to make a system completely transparent and users, generally, are aware that they are dealing with a distributed system. You may therefore decide that it is best to expose the distribution to users. They can then be prepared for some of the consequences of distribution such as network delays, remote node failures, etc.

Open distributed systems are systems that are built according to generally accepted standards. This means that components from any supplier can be integrated into the system and can interoperate with the other system components. At the networking level, openness is now taken for granted with systems conforming to Internet protocols but at the component level, openness is still not universal. Openness implies that system components can be independently developed in any programming language and, if these conform to standards, they will work with other components.

The CORBA standard (Pope, 1997) developed in the 1990s, was intended to achieve this but this never achieved a critical mass of adopters. Rather, many companies chose to develop systems using proprietary standards for components from companies such as Sun and Microsoft. These provided better implementations and support software and better long-term support for industrial protocols.

Web service standards (discussed in Chapter 19) for service-oriented architectures were developed to be open standards. However, there is significant resistance to these standards because of their perceived inefficiency. Some developers of service-based systems have opted instead for so-called RESTful protocols as these have an inherently lower overhead than web service protocols.

The scalability of a system reflects its ability to deliver a high quality of service as demands on the system increase. Neuman (1994) identifies three dimensions of scalability:

1. *Size* It should be possible to add more resources to a system to cope with increasing numbers of users.
2. *Distribution* It should be possible to geographically disperse the components of a system without degrading its performance.
3. *Manageability* It should be possible to manage a system as it increases in size, even if parts of the system are located in independent organizations.

In terms of size, there is a distinction between scaling up and scaling out. Scaling up means replacing resources in the system with more powerful resources. For example, you may increase the memory in a server from 16 GB to 64 GB. Scaling out means adding additional resources to the system (e.g., an extra web server to work alongside an existing server). Scaling out is often more cost effective than scaling up but usually means that the system has to be designed so that concurrent processing is possible.

I have discussed general security issues and issues of security engineering in Part 2 of this book. However, when a system is distributed, the number of ways that the system may be attacked is significantly increased, compared to centralized systems. If a part of the system is successfully attacked then the attacker may be able to use this as a 'back door' into other parts of the system.

The types of attacks that a distributed system must defend itself against are the following:

1. Interception, where communications between parts of the system are intercepted by an attacker so that there is a loss of confidentiality.
2. Interruption, where system services are attacked and cannot be delivered as expected. Denial of service attacks involve bombarding a node with illegitimate service requests so that it cannot deal with valid requests.
3. Modification, where data or services in the system are changed by an attacker.
4. Fabrication, where an attacker generates information that should not exist and then uses this to gain some privileges. For example, an attacker may generate a false password entry and use this to gain access to a system.